



UNIVERSITÀ DEGLI STUDI DI FIRENZE

SCUOLA DI INGEGNERIA - DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Tesi di Laurea Triennale in Ingegneria Informatica

**SVILUPPO, IMPLEMENTAZIONE E ANALISI
COMPARATIVA
DELL'ALGORITMO DI PURE PURSUIT IN AMBIENTI
DI
SIMULAZIONE PER LA GUIDA AUTONOMA**

Candidato

Marco Tetsuya Stella

Relatore

Prof. Giorgio Battistelli

N. Matricola

7133891

Anno Accademico 2025/2026

Indice

1	Introduzione	1
2	Fondamenti teorici e modello del sistema	3
2.1	Introduzione di capitolo	3
2.2	Modello cinematico del robot mobile	3
2.3	Sensore LiDAR	4
2.4	Localizzazione tramite odometria e algoritmo ICP	6
2.5	Strategie per la riduzione del drift	7
2.5.1	Criteri di accettazione dell'allineamento	7
2.5.2	Contenimento della dimensione della mappa tramite voxel down-sampling	7
2.5.3	<i>Loop Closure</i> basato su <i>Keyframe</i>	8
3	Implementazione del sistema di simulazione	9
3.1	Introduzione di capitolo	9
3.2	Architettura del simulatore	9
3.2.1	Infrastruttura di Base Preesistente	10
3.2.2	Moduli nuovi sviluppati per la Tesi	10
3.3	Generazione di percorsi e percorsi predefiniti per i test	12
3.3.1	Il modulo <code>path_generator.py</code>	12
3.3.2	Il modulo <code>prefabricated_paths.py</code>	16
3.4	Generazione di ambienti predefiniti per i test	20
3.4.1	Il modulo <code>environment_presets_pure_pursuit.py</code>	20
3.5	Simulazione dell'Odometria Rumorosa	26
3.5.1	Il modulo <code>noisy_odometry.py</code>	26
3.6	Esecuzione degli esperimenti, visualizzazione e salvataggio dei risultati .	28
3.6.1	Il modulo <code>main_pure_pursuit.py</code>	28
3.6.2	Il modulo <code>visualizer_pure_pursuit.py</code>	28
4	Algoritmo Pure Pursuit ed esecuzione di esso	30
4.1	Introduzione di capitolo	30

4.2	L'algoritmo Pure Pursuit	30
4.3	Il modulo pure-pursuit.py	31
4.4	Il modulo pure-pursuit_simulation.py	39
5	Valutazione sperimentale	43
5.1	Introduzione di capitolo	43
5.2	Caratterizzazione dei parametri per type di percorso	43
5.3	Metriche di valutazione	46
5.4	Configurazione degli esperimenti	46
5.4.1	Esperimento <i>straight long</i>	46
5.4.2	Esperimento <i>contapassi</i>	50
5.4.3	Esperimento di tipo <i>pista 3 (due giri)</i>	57
6	Conclusioni e sviluppi futuri	64
6.1	Riassunto degli esperimenti	64
6.2	Conclusioni	66
6.3	Sviluppi futuri	66

Capitolo 1

Introduzione

Questo lavoro di tesi si concentra sullo sviluppo, l'implementazione e l'analisi sperimentale dell'algoritmo di inseguimento della traiettoria *Pure Pursuit*. La ricerca si pone in diretta continuità con due precedenti tesi: "*Simulazione di robot mobile e localizzazione con algoritmo ICP su scansioni LiDAR*" e "*Mappatura e riduzione del drift in sistemi 2D basati su LiDAR e ICP*".

Nelle tesi precedenti è stato sviluppato un sistema per la localizzazione di un robot mobile in ambienti bidimensionali, fondato sulla fusione dei dati provenienti dall'odometria del robot e dal sensore LiDAR tramite l'algoritmo *Iterative Closest Point* (ICP).

Inizialmente il sensore LiDAR va ad emettere degli impulsi laser i quali hanno il compito di scansionare l'ambiente, la scansione avviene misurando il tempo di volo che i laser impiegano per essere emessi, riflettersi sugli ostacoli e ritornare al sensore.

Dopodiché l'algoritmo ICP utilizzando tali scansioni va a stimare la rototraslazione necessaria per allineare la scansione corrente con una mappa globale accumulata, tale stima rappresenta lo spostamento del robot tra un passo e un altro. In aggiunta è stato implementato un modulo di *Loop Closure*, che si attiva ogni volta che il robot ripassa per una posizione già conosciuta, in tal caso il sistema interviene correggendo la posa precedentemente stimata andando ad azzerare gli eventuali errori di *drift* che ha commesso nel tempo.

La presente tesi estende le funzionalità sviluppate introducendo un sistema di controllo in grado di far seguire al robot una traiettoria desiderata tramite l'algoritmo *Pure Pursuit*, valutandone le prestazioni e il comportamento al variare delle condizioni operative.

L'algoritmo *Pure Pursuit* si basa sulla conoscenza della posizione stimata del robot, calcolata dai sensori e posizionata convenzionalmente nel suo centro geometrico. A partire da tale centro, si definisce un'area di visione circolare avente come raggio la distanza di *look-ahead*. L'algoritmo individua il punto di intersezione tra questa

circonferenza e il percorso pianificato, calcolando di conseguenza i comandi di velocità angolare necessari a indirizzare il robot verso tale punto.

Come verifica del corretto funzionamento sono stati creati dodici tracciati di prova, ogni tracciato presenta tre diversi tipi di ambiente. Questo permette di andare a studiare come i principali parametri vanno ad influenzare il comportamento del robot. I parametri in questione sono: il tipo di odometria (ideale o rumorosa), l'utilizzo o meno della funzionalità di *Loop Closure*, la massima portata (*range*) dei raggi LiDAR il cui valore viene scelto in base all'ambiente di test e il valore della distanza di *look-ahead* del *Pure Pursuit*.

L'elaborato è organizzato nei seguenti capitoli:

Capitolo 2 – Fondamenti teorici e modello del sistema: richiama i concetti teorici fondamentali sviluppati nelle tesi precedenti (odometria, LiDAR, ICP e *Loop Closure*), indispensabili per la comprensione del lavoro svolto.

Capitolo 3 – Implementazione del sistema di simulazione: descrive nel dettaglio l'architettura software del simulatore realizzato, analizzando le singole componenti e la loro interazione.

Capitolo 4 – Algoritmo Pure Pursuit ed esecuzione di esso: descrive nel dettaglio il funzionamento teorico dell'algoritmo *Pure Pursuit*, analizzando l'implementazione del codice software e la sua integrazione all'interno dell'architettura di simulazione per l'esecuzione dei test.

Capitolo 5 – Valutazione sperimentale: espone e discute i risultati ottenuti nei diversi scenari di test, confrontando le metriche di errore al variare dei parametri di localizzazione e di *look-ahead*.

Capitolo 6 – Conclusioni e sviluppi futuri: sintetizza i risultati principali emersi dalla sperimentazione e propone possibili estensioni e miglioramenti per ricerche future.

Capitolo 2

Fondamenti teorici e modello del sistema

2.1 Introduzione di capitolo

Come anticipato nell'Introduzione, il presente lavoro di tesi si pone in diretta continuità con le ricerche svolte nei due progetti precedenti: *"Simulazione di robot mobile e localizzazione con algoritmo ICP su scansioni LiDAR"* e *"Mappatura e riduzione del drift in sistemi 2D basati su LiDAR e ICP"*.

Per semplificare la comprensione degli argomenti trattati, in questo capitolo vengono presentati brevemente i fondamenti teorici e i modelli rappresentativi utilizzati nel sistema, i quali sono stati trattati nelle tesi precedenti.

2.2 Modello cinematico del robot mobile

Il robot viene descritto mediante il modello cinematico dell'uniciclo, in cui il vettore degli ingressi di controllo è definito da due grandezze: la velocità lineare v e la velocità angolare ω . Dal momento che il modello assume l'assenza di slittamento extended e vincola il moto esclusivamente lungo la direzione di avanzamento del robot, tale modello risulta ideale per la rappresentazione di sistemi non olonomi (ovvero sistemi caratterizzati da vincoli anolonomi sulle velocità che non possono essere integrati per ottenere vincoli sulle sole posizioni, il che implica l'impossibilità di traslare direttamente in qualsiasi direzione del piano senza variare l'orientamento del veicolo).

Il robot si muove su un piano bidimensionale e la sua configurazione (ovvero la sua posa nel piano) è rappresentata dal seguente vettore di stato:

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \alpha \end{bmatrix} \quad (2.1)$$

dove x e y indicano le coordinate cartesiane della posizione del robot sul piano, mentre α rappresenta il suo angolo di orientamento rispetto all'asse x .

Di conseguenza, le equazioni cinematiche del sistema continuo sono espresse da:

$$\begin{cases} \dot{x} = v \cos(\alpha) \\ \dot{y} = v \sin(\alpha) \\ \dot{\alpha} = \omega \end{cases} \quad (2.2)$$

Per consentire l'implementazione all'interno dell'ambiente di simulazione, tali equazioni vengono discretizzate nel tempo mediante il metodo di Eulero in avanti:

$$\begin{cases} x_{k+1} = x_k + \Delta t \cdot v_k \cos(\alpha_k) \\ y_{k+1} = y_k + \Delta t \cdot v_k \sin(\alpha_k) \\ \alpha_{k+1} = \alpha_k + \Delta t \cdot \omega_k \end{cases} \quad (2.3)$$

Le equazioni sopra riportate consentono di aggiornare lo stato del sistema dal passo temporale corrente k al successivo $k + 1$, in cui Δt rappresenta l'intervallo di campionamento (passo temporale) della simulazione.

Viene posta grande importanza nella scelta riguardo al valore di Δt il quale è compreso tra 0.05 s e 0.1 s; questa scelta accoppiata insieme ad una scelta della velocità di avanzamento contenuta, permette di garantire un'adeguata precisione del modello e stabilità di controllo. Infatti andando a scegliere un valore di Δt troppo grande si introducono degli errori significativi riguardanti la discretizzazione e l'approssimazione numerica, mentre con un valore troppo piccolo andremmo ad aumentare il carico computazionale senza andare ad ottenere un miglioramento in termini di accuratezza.

Per l'intero lavoro di tesi, tutte le simulazioni sono state eseguite impostando un valore fisso di $\Delta t = 0.05$ s.

2.3 Sensore LiDAR

Uno degli elementi fondamentali delle precedenti tesi è il sensore LiDAR (*Light Detection and Ranging*). Nel nostro modello, si ipotizza che il sensore sia posizionato esattamente al centro del robot. Il LiDAR opera emettendo un numero variabile di raggi laser (nel nostro caso specifico, 360) che si riflettono sugli oggetti presenti nell'ambiente bidimensionale; rilevando tali riflessioni e conoscendo la velocità di propagazione della luce, il sensore è in grado di calcolare la distanza dai singoli ostacoli, fornendo così un'informazione discreta sulla struttura dello spazio circostante.

L'operatività del sensore viene simulata mediante la tecnica del *ray-casting*. Tale algoritmo permette di simulare la propagazione dei raggi laser nell'ambiente, andando per ogni raggio a calcolare il punto di intersezione con l'ostacolo che risulta più vicino al robot. L'insieme di questi punti va a generare una *point cloud* (nuvola di punti), in cui ogni punto rappresenta le coordinate in cui il raggio laser ha impattato con un ostacolo.

I parametri fondamentali che caratterizzano la scansione e la relativa *point cloud* comprendono il numero di raggi (N), che determina la risoluzione angolare della misurazione, l'apertura angolare (FOV, o *Field of View*), la quale definisce l'ampiezza del campo visivo del sensore, e infine la portata massima ($r_{\max}$), che rappresenta la distanza limite oltre la quale gli ostacoli non vengono più rilevati.

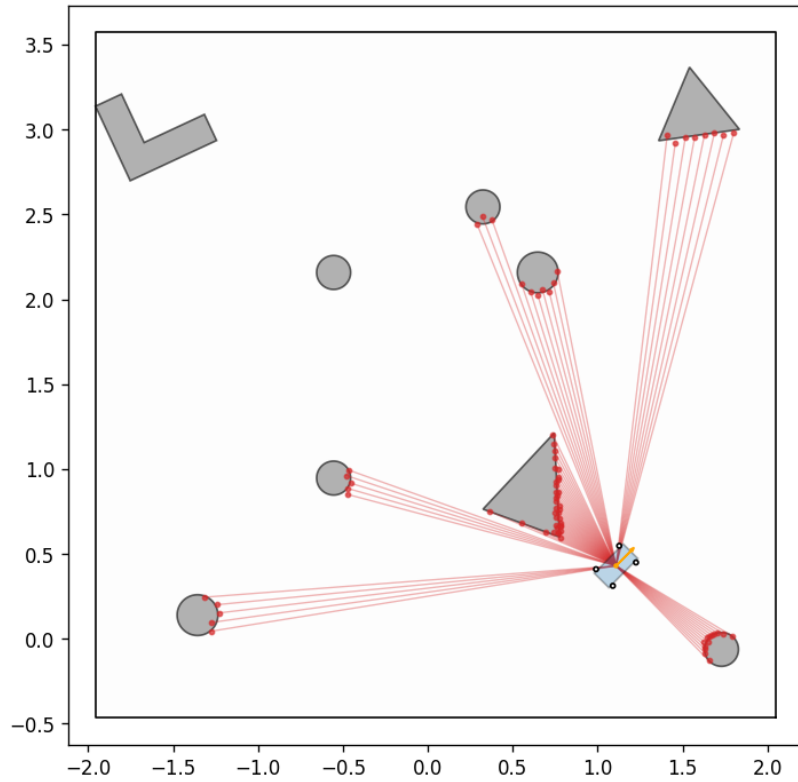


Figura 2.1: Simulazione del sensore LiDAR tramite la tecnica di *ray-casting*. I punti rossi rappresentano la *point cloud* generata dall'intersezione dei raggi con gli ostacoli.

2.4 Localizzazione tramite odometria e algoritmo ICP

Per poter applicare l'algoritmo di *Pure Pursuit* è fondamentale conoscere con precisione la posizione del robot all'interno dell'ambiente. A tale scopo, ci si affida a una stima ottenuta dalla combinazione tra la misurazione odometrica e l'algoritmo ICP (*Iterative Closest Point*).

Tale integrazione si rende necessaria poiché, in scenari reali, l'utilizzo della sola odometria comporta un progressivo accumulo di errore noto come *drift* odometrico.

La causa del fenomeno risiede nelle imprecisioni del modello cinematico, agli errori di discretizzazione temporale e alle incertezze sui comandi di controllo, i quali provocano una divergenza crescente tra la posa stimata e quella reale del veicolo.

Per ovviare a questo problema ci affidiamo alle scansioni ottenute tramite il sensore LiDAR le quali permettono di correggere la stima odometrica attraverso l'algoritmo ICP. Lo scopo dell'algoritmo ICP (*Iterative Closest Point*) consiste nel riuscire ad allineare due nuvole di punti andando ad individuare la trasformazione rigida ottimale — definita da una matrice di rotazione $\mathbf{R} \in \mathbb{R}^{2 \times 2}$ e da un vettore di traslazione $\mathbf{t} \in \mathbb{R}^2$ — la quale permette di sovrapporre la nuvola sorgente (*source*) alla nuvola target, minimizzando la distanza tra le corrispondenze.

Il processo è di natura iterativa e ad ogni passo esegue due fasi principali:

1. **Associazione delle corrispondenze (*Nearest Neighbor Matching*):** Per ciascun punto della nuvola sorgente viene individuato il punto più vicino all'interno della nuvola target, generando così un insieme di coppie di punti associati.
2. **Stima della trasformazione:** Si calcolano $\mathbf{R}$ e $\mathbf{t}$ in modo da minimizzare l'errore quadratico medio (*Root Mean Square Error*, RMSE) tra i punti corrispondenti. Nel caso della variante *point-to-point*, la soluzione analitica si ottiene mediante la decomposizione ai valori singolari (*Singular Value Decomposition*, SVD).

L'algoritmo arresta l'esecuzione quando la variazione dell'RMSE tra iterazioni successive diventa inferiore a una soglia di tolleranza prefissata.

Essendo un metodo di ottimizzazione locale, la convergenza dell'ICP dipende fortemente dalla bontà della stima iniziale. Per questa ragione, la stima odometrica viene impiegata come *initial guess* (punto di partenza), evitando che l'algoritmo rimanga bloccato in minimi locali indesiderati.

Infine, l'allineamento può avvenire confrontando due scansioni consecutive (*scan-to-scan*) oppure confrontando la scansione corrente con una mappa globale dell'ambiente già costruita (*scan-to-map*). Per questa tesi è stato adottato l'approccio *scan-to-map*, in quanto garantisce una maggiore stabilità e riduce la deriva nella stima della posa del robot.

2.5 Strategie per la riduzione del drift

Per ridurre ulteriormente il *drift* e garantire la stabilità della stima nel tempo, vengono adottate le seguenti strategie.

2.5.1 Criteri di accettazione dell'allineamento

Prima di andare ad applicare l'aggiornamento della mappa e della posa, tramite la stima della trasformazione calcolata dall'algoritmo ICP, ci accertiamo che il numero delle corrispondenze valide risulti superiore a una soglia minima e che il valore di RMSE finale calcolato risulti sufficientemente basso.

Nel caso in cui il tentativo di allineamento fallisca non rispettando i criteri esposti, la stima della posa del robot viene affidata unicamente alla predizione odometrica, andando ad evitare l'uso di correzioni errate.

2.5.2 Contenimento della dimensione della mappa tramite voxel downsampling

L'integrazione continua delle scansioni causa una crescita progressiva della nuvola di punti complessiva, con un conseguente incremento del carico computazionale e una forte ridondanza informativa. Per ovviare a questo problema, quando la mappa supera una determinata soglia di punti, viene applicato un filtro di *voxel downsampling*.

Tale tecnica riduce la densità della *point cloud* preservandone la struttura geometrica essenziale; in questo modo si bilanciano l'accuratezza della rappresentazione e l'efficienza computazionale, garantendo la stabilità dell'ICP anche lungo traiettorie di lunga durata e contenendo indirettamente il *drift*.

Dal momento che le funzionalità native di *voxel downsampling* fornite dalla libreria *Open3D* sono progettate per operare nello spazio tridimensionale, è stata implementata una funzione dedicata per estenderne l'utilizzo ad ambienti bidimensionali. Come mostrato nel Listing 2.1, la procedura converte temporaneamente i punti 2D aggiungendo una coordinata $z = 0$, applica l'algoritmo di campionamento integrato in *Open3D* e, infine, proietta i punti estratti nuovamente nel piano 2D scartando la dimensione ausiliaria.

```
1 def voxel_downsample_2d(points: np.ndarray, voxel_size: float = 0.03) -> np.  
  ndarray:  
2     #Riduce la densità dei punti 2D raggruppandoli in una griglia di voxel.  
3     if len(points) == 0:  
4         return points  
5     # Conversione temporanea dei punti da 2D a 3D (asse z = 0)  
6     points_3d = np.hstack([points, np.zeros((points.shape[0], 1))])
```

```

7      # Creazione del tipo PointCloud compatibile con Open3D
8      pcd = o3d.geometry.PointCloud()
9      pcd.points = o3d.utility.Vector3dVector(points_3d)
10     # Applicazione del Voxel Down Sampling nativo di Open3D
11     pcd_down = pcd.voxel_down_sample(voxel_size)
12     # Estrazione dei punti campionati e riproiezione nello spazio 2D
13     return np.asarray(pcd_down.points)[: , :2]

```

Listing 2.1: Implementazione del *voxel downsampling* per nuvole di punti 2D.

2.5.3 *Loop Closure* basato su *Keyframe*

L'utilizzo dei *keyframe* assieme al meccanismo di *loop closure* offre un contributo importante nel contenere il *drift* correggendo l'errore accumulato nel tempo. I *keyframe* sono tutte le pose che vengono considerate significative, alle quali vengono associate le rispettive scansioni d'ambiente. Il sistema seleziona un nuovo *keyframe* solo se viene rilevato uno spostamento rilevante in termini di distanza percorsa o variazione angolare rispetto all'ultimo *keyframe*, così da evitare di avere informazioni ridondanti.

L'insieme dei *keyframe* viene utilizzato dal modulo di *loop closure*, questo avviene quando un robot si trova a distanza di tempo in un punto geometricamente vicino a un *keyframe* passato. In tal caso avviene una verifica di tipo ICP. In cui si va a verificare se l'allineamento prodotto dall'ICP rispetta delle condizioni riguardanti il numero delle corrispondenze, l'errore RMSE e la misura di *fitness*, la quale viene definita come il rapporto di sovrapposizione tra le due point cloud. Nel caso in cui il controllo dia un esito positivo, avviene la correzione della posa andando ad azzerare l'errore residuo accumulato nel tempo.

Capitolo 3

Implementazione del sistema di simulazione

3.1 Introduzione di capitolo

Nel presente capitolo viene descritta l'architettura software del simulatore sviluppato, analizzando nel dettaglio le singole componenti che costituiscono il sistema.

Prima di esaminare i moduli software e il ciclo di simulazione, è opportuno introdurre brevemente il principio di funzionamento del *Pure Pursuit*. Tale algoritmo rappresenta il nucleo concettuale dell'intero lavoro di tesi ed è fondamentale comprenderne la logica di base per identificare il ruolo dei diversi elementi dell'architettura.

Il *Pure Pursuit* è un algoritmo di inseguimento di traiettoria (*path tracking*) che guida un robot differenziale lungo un percorso prestabilito. Il principio cardine si basa sulla definizione di una distanza di mira, detta *look-ahead distance* (L_d). Geometricamente, la L_d definisce una circonferenza di raggio pari a tale distanza, centrata sulla posizione corrente del robot. L'algoritmo calcola l'intersezione tra questa circonferenza e la traiettoria da seguire, individuando un punto di mira (*look-ahead point*). Successivamente, determina il raggio di curvatura necessario a raggiungere tale punto mediante un arco di circonferenza, inviando i corrispondenti comandi di velocità al robot.

3.2 Architettura del simulatore

L'architettura del simulatore è stata progettata seguendo un approccio modulare per garantire il principio di separazione delle responsabilità, scrivendo così un codice più sicuro e meno soggetto ad errori.

L'architettura software del simulatore si articola in una serie di moduli interconnessi, suddivisi tra l'infrastruttura preesistente (ereditata da precedenti lavori di tesi) e i nuovi

moduli sviluppati specificamente per questa tesi per l'esecuzione del pure pursuit e la valutazione sperimentale dei risultati ottenuti.

3.2.1 Infrastruttura di Base Preesistente

I moduli preesistenti costituiscono l'infrastruttura fisica, sensoriale e di localizzazione della piattaforma di simulazione:

- `robot.py`: definisce la classe `Robot` per la modellazione cinematica di un veicolo differenziale. Gestisce lo stato bidimensionale del robot (x, y, θ) , consentendo l'impostazione dei comandi di velocità lineare v e angolare ω . L'aggiornamento temporale della posa avviene tramite integrazione numerica discreta (schema di Eulero esplicito) con contestuale normalizzazione dell'angolo di orientamento θ nell'intervallo $[-\pi, \pi]$.
- `lidar.py`: implementa la simulazione del sensore LiDAR tramite tecniche di *raycasting*, generando nuvole di punti 2D (*point cloud*) in base alla posa corrente del robot e alla disposizione degli ostacoli nell'ambiente.
- `environment.py`: rappresenta lo spazio operativo e le geometrie degli ostacoli, fornendo i metodi per il calcolo delle intersezioni e per la verifica delle collisioni.
- `simulator.py`: contiene la classe `Simulator`, la quale gestisce l'orchestrazione complessiva dell'avanzamento della fisica del sistema e l'acquisizione dei dati (*data logging*), memorizzando la cronologia delle pose reali assunte dal veicolo e i comandi effettivamente applicati.
- `icp.py`: implementa l'algoritmo *Iterative Closest Point* (ICP) e le relative routine ausiliarie per la stima della posa e la localizzazione basata su scansioni sensoriali.
- `loop_closure.py`: fornisce strutture dati e strumenti matematici per la gestione del *loop closure* e la correzione della deriva di localizzazione mediante una rete di *Keyframe* allineati tramite ICP.
- `visualizer.py`: fornisce le primitive grafiche di base per il rendering; in questo lavoro di tesi viene riutilizzato all'interno dei moduli `visualizer_pure_pursuit.py` e `main2.py` sfruttando specificamente la funzione `draw_robot` per il disegno dinamico del veicolo nelle sequenze animate.

3.2.2 Moduli nuovi sviluppati per la Tesi

A supporto dell'algoritmo di *Pure Pursuit*, dell'analisi di robustezza e della generazione sperimentale di scenari, sono stati progettati ed estesi i seguenti moduli originali:

- `path_generator.py`: fornisce una suite di generatori geometrici per la creazione discreta di traiettorie e percorsi espresso sotto forma di matrici di $N \times 2$ punti di riferimento (*waypoint*).
- `prefabricated_paths.py`: implementa un registro centralizzato di percorsi predefiniti (*preset*) pronti all'uso per i benchmark di test.
- `environment_presets_pure_pursuit.py`: genera ambienti di simulazione complessi posizionando gli ostacoli in modo deterministico attraverso l'uso della sequenza quasi-casuale di Halton. Garantisce un margine di sicurezza espresso come distanza di *clearance* minima attorno alle traiettorie di riferimento e include strumenti per l'anteprima visuale delle mappe.
- `noisy_odometry.py`: simula il comportamento imperfetto di un sensore odometrico reale iniettando due componenti di disturbo distinte: un rumore di fondo gaussiano continuo, proporzionale alle velocità del robot, ed eventi sporadici ad alta intensità (es. slittamenti delle ruote o perdite di aderenza) modellate mediante disturbi impulsivi casuali su velocità lineare ed angolare.
- `pure_pursuit.py`: contiene il nucleo dell'algoritmo di guida. Calcola i comandi di velocità angolare ω (mantenendo costante la velocità lineare v) necessari per seguire la traiettoria di riferimento, individuando il *look-ahead point* e risolvendo la geometria di curvatura locale.
- `pure_pursuit_simulation.py`: coordina l'esecuzione delle simulazioni di *Pure Pursuit*, consentendo la configurazione parametrica delle condizioni di prova (presenza o assenza di rumore odometrico, attivazione o disattivazione dei moduli di *loop closure* e ICP).
- `visualizer_pure_pursuit.py`: interfaccia grafica avanzata ed interattiva che permette la visualizzazione animata del ciclo di simulazione e il controllo passo-passo dell'esecuzione. Calcola le metriche di errore medio tra la traiettoria ideale e quella realmente percorsa, genera grafici delle traiettorie e diagrammi a barre degli errori (anch'essi consultabili da interfaccia) e gestisce l'esportazione automatica della reportistica grafica su disco.
- `main2.py`: esegue la simulazione dell'algoritmo *Pure Pursuit* sul tracciato selezionato, mostrando l'animazione in tempo reale a schermo e salvando il filmato in formato MP4 all'interno della cartella `video_pure_pursuit`.
- `main_pure_pursuit.py`: punto di ingresso principale dell'applicazione che integra le routine di simulazione da `pure_pursuit_simulation.py` con la classe

`InteractiveVisualizer` definita in `visualizer_pure_pursuit.py`, permettendo l'avvio e la configurazione rapida delle sessioni sperimentali.

Di seguito vengono analizzati nel dettaglio i principali file di codice sviluppati per il lavoro di tesi. Si precisa che la trattazione approfondita dei moduli `pure_pursuit.py` e `pure_pursuit_simulation.py` verrà rinviata al capitolo successivo, interamente dedicato alla formulazione teorica, all'implementazione e all'orchestrazione del sistema di guida basato su Pure Pursuit.

3.3 Generazione di percorsi e percorsi predefiniti per i test

3.3.1 Il modulo `path_generator.py`

La generazione delle traiettorie di riferimento è affidata alla classe `PathGenerator`, definita all'interno del modulo `path_generator.py`. La classe mette a disposizione un insieme di metodi statici, ciascuno progettato per generare una specifica tipologia di percorso:

- `straight_line`: genera una traiettoria rettilinea interpolando discretamente il segmento tra due punti di controllo (x_1, y_1) e (x_2, y_2) .
- `circle`: genera un percorso circolare calcolando la sequenza dei punti lungo una circonferenza, sulla base del raggio e delle coordinate del centro forniti in ingresso.
- `slalom`: genera un tracciato sinusoidale a forma di "S", definibile in termini di ampiezza e frequenza dell'onda.
- `figure_eight`: costruisce una traiettoria complessa a forma di otto mediante equazioni parametriche trigonometriche (utilizzando una frequenza doppia sull'asse Y rispetto all'asse X).
- `custom_polygon`: genera una traiettoria spezzata o chiusa (es. triangoli, rettangoli o poligoni arbitrari) collegando sequenzialmente una lista di vertici fornita dall'utente.

Ciascun metodo restituisce in output una matrice NumPy di dimensione $N \times 2$, dove N rappresenta il numero totale di *waypoint* discretizzati: la prima colonna (colonna 0) memorizza le coordinate X espressa in metri, mentre la seconda colonna (colonna 1) contiene le corrispondenti coordinate Y .

Tra i metodi implementati, quelli maggiormente interessanti sia dal punto di vista sperimentale sia sotto l'aspetto algoritmico e di codice sono le seguenti.

Generazione della Traiettoria a Slalom

Il metodo statico `slalom` consente di generare un percorso oscillatorio di tipo sinusoidale che si sviluppa lungo l'asse orizzontale X .

La coordinata trasversale $y(x)$ viene calcolata in funzione della coordinata longitudinale $x \in [x_{\text{start}}, x_{\text{end}}]$ tramite la seguente relazione matematica:

$$y(x) = A \cdot \sin\left(2\pi \cdot \frac{x}{\lambda}\right) \quad (3.1)$$

dove:

- A rappresenta l'ampiezza dell'onda (`amplitude`), che determina lo scostamento massimo del veicolo lungo l'asse Y rispetto all'asse di avanzamento;
- λ rappresenta la lunghezza d'onda (`wavelength`), ovvero la distanza metrica necessaria lungo l'asse X per completare un'oscillazione sinusoidale intera. Scalare la posizione x rispetto a λ assicura che per $x = \lambda$ l'argomento della funzione seno sia esattamente pari a 2π , completando così un ciclo periodico completo.

Di seguito viene riportata l'implementazione del metodo in linguaggio Python:

```
1 def slalom(start_x: float, end_x: float, amplitude: float, wavelength: float
2   , num_points: int = 150) -> np.ndarray:
3     """
4     Genera un percorso a forma di onda sinusoidale che si sviluppa lungo l'
5     asse X.
6     Args:
7         start_x: Coordinata X di inizio dello slalom.
8         end_x: Coordinata X di fine dello slalom.
9         amplitude: Ampiezza dell'onda (quanto si sposta lateralmente sull'
10        asse Y).
11        wavelength: Lunghezza d'onda (distanza percorsa su X per completare
12        un'oscillazione).
13        num_points: Numero di punti da generare.
14    Returns:
15        Array numpy di forma (num_points, 2) che descrive lo slalom.
16    """
17    # Genera uniformemente le coordinate di avanzamento lungo l'asse X
18    xs = np.linspace(start_x, end_x, num_points)
19
20    # Calcola le coordinate Y applicando la funzione seno.
21    # Quando x = wavelength, l'argomento diventa 2*pi, completando
    esattamente
22    1 onda intera
```

```

23     ys = amplitude * np.sin(2 * np.pi * xs / wavelength)
24
25     # Impila le coordinate X e Y per colonna creando una matrice (num_points
    , 2)
26     return np.stack([xs, ys], axis=1)

```

Listing 3.1: Implementazione della traiettoria a slalom con funzione sinusoidale.

Generazione di Traiettorie Critiche per i Test di Prestazione

Un altro metodo di notevole rilievo, sia per la flessibilità implementativa nel codice sia per il valore sperimentale, è la funzione `custom_polygon`. Tale metodo consente di costruire percorsi spezzati o chiusi interpolando sequenzialmente una lista di vertici arbitrari forniti dall'utente.

Tra le applicazioni più importanti di questa funzione figura la creazione di **percorsi caratterizzati da curve molto strette e angoli a gomito**. Questa tipologia di geometrie è risultata fondamentale per sollecitare il sistema di guida, consentendo di valutare e testare il comportamento dell'algoritmo di *Pure Pursuit* al variare della *look-ahead distance*.

Dal punto di vista dell'implementazione, la funzione gestisce l'interpolazione lineare a tratti evitando sovrapposizioni di punti adiacenti (tramite la clausola `endpoint=False`) e garantendo la chiusura esatta della traiettoria mediante la concatenazione delle matrici dei singoli segmenti (`np.vstack`).

Di seguito si riporta il codice sorgente del metodo:

```

1 def custom_polygon(vertices: list, points_per_segment: int = 30, close_loop:
2   bool = True) -> np.ndarray:
3     """
4     Genera un percorso spezzato collegando linearmente una lista di vertici
5     forniti.
6     Args:
7         vertices: Lista di tuple o coordinate [(x1, y1), (x2, y2), ...] dei
8         vertici.
9         points_per_segment: Numero di waypoint da interpolare per ogni lato
10        del poligono.
11        close_loop: Se True, aggiunge un segmento che riporta l'ultimo
12        vertice al primo.
13    Returns:
14        Array numpy di forma (N, 2) contenente tutti i segmenti interpolati
15        uniti.
16    """
17    # Controllo di sicurezza: serve un punto di inizio e uno di fine per

```

```

18     # fare un percorso
19     if len(vertices) < 2:
20         raise ValueError("Sono necessari almeno 2 vertici per generare un
21         percorso.")
22
23     # Copia la lista per non modificare i dati originali
24     pts = list(vertices)
25
26     # Se il loop va chiuso e l'ultimo punto non è già uguale al primo,
27     # duplica il primo punto alla fine
28     if close_loop and pts[-1] != pts[0]:
29         pts.append(pts[0])
30
31     path_segments = []
32     # Cicla attraverso tutte le coppie di vertici adiacenti
33     for i in range(len(pts) - 1):
34         p_start = pts[i]
35         p_end = pts[i + 1]
36
37         # Interpoli i valori della X per questo specifico lato del poligono.
38         # endpoint=False evita che l'ultimo punto di un lato si sovrapponga
39         # al primo punto del lato successivo.
40         segment_x = np.linspace(p_start[0], p_end[0], points_per_segment,
41         endpoint=False)
42
43         # Interpoli i valori della Y per lo stesso lato
44         segment_y = np.linspace(p_start[1], p_end[1], points_per_segment,
45         endpoint=False)
46
47         # Salva i punti di questo lato nella lista dei segmenti
48         path_segments.append(np.stack([segment_x, segment_y], axis=1))
49
50     # Aggiunge esplicitamente l'ultimissimo punto per garantire che il
51     traguardo (o la chiusura) sia esatto
52     path_segments.append(np.array([pts[-1]]))
53
54     # np.vstack concatena verticalmente tutte le matrici (tutti i lati) in
55     un
56     unico lungo array N x 2
57     return np.vstack(path_segments)

```

Listing 3.2: Implementazione della funzione `custom_polygon` per la generazione di percorsi spezzati e poligonali.

3.3.2 Il modulo `prefabricated_paths.py`

Il modulo `prefabricated_paths.py` è stato progettato per organizzare e mettere a disposizione un insieme di scenari di test preconfigurati (*preset*). Sfruttando le funzioni generatrici fornite da `path_generator.py`, il modulo definisce dei percorsi con parametri chiave già impostati deterministicamente (quali coordinate dei vertici, lunghezze dei tratti, raggi di curvatura e frequenze).

I singoli tracciati sono incapsulati all'interno di metodi statici dedicati. L'interazione con l'insieme dei percorsi disponibili è gestita in maniera flessibile e centralizzata mediante due metodi di classe (*classmethod*):

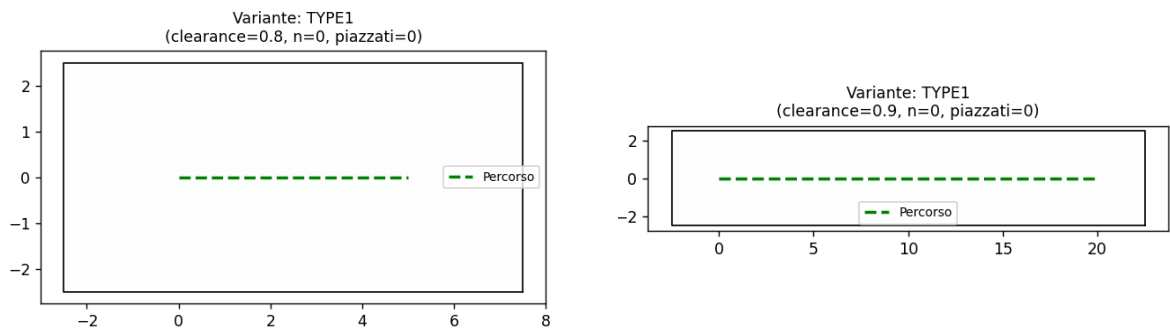
- `list_presets()`: restituisce l'elenco completo di tutte le stringhe identificative dei percorsi configurati all'interno del registro;
- `get_preset(name:str)`: consente l'acquisizione e il caricamento dinamico della matrice di *waypoint* corrispondente al percorso desiderato, semplicemente fornendone il nome come argomento di tipo stringa.

Questa struttura garantisce una gestione modulare dei percorsi di prova, facilitando il riuso dei test in diverse configurazioni sperimentali ed evitando la ridondanza di codice nella definizione delle traiettorie. Ci sono dodici percorsi di prova i quali sono definiti tramite i seguenti metodi:

- `get_straight_short`: genera un segmento di retta di lunghezza ridotta (5 metri), utilità per test basilari di avanzamento rettilineo.
- `get_straight_long`: definisce un rettilineo esteso di 20 metri per valutare la stabilità e la convergenza del controller su distanze maggiori.
- `get_circle_medium`: costruisce una traiettoria circolare con raggio di 3 metri.
- `get_circle_large`: genera una traiettoria circolare di ampiezza maggiore con raggio pari a 6 metri.
- `get_tight_slalom`: definisce un percorso sinusoidale stretto lungo 15 metri, con un'ampiezza di 0,8 metri e una lunghezza d'onda di 3 metri, ideato per testare la prontezza di risposta nei cambi di direzione ravvicinati.

- `get_wide_slalom`: genera uno slalom ampio lungo 20 metri, caratterizzato da un'ampiezza di 2 metri e una lunghezza d'onda di 8 metri.
- `get_standard_eight`: costruisce un tracciato a forma di otto con semi-ampiezza longitudinale (lungo X) di 6 metri e semi-ampiezza trasversale (lungo Y) di 3 metri.
- `get_square_circuit`: definisce un circuito quadrato regolare avente lato di 8 metri, introducendo quattro cambi di direzione netti a 90° .
- `get_pista_1`: implementa un circuito chiuso composto da curve a variazione di curvatura complessa.
- `get_pista_2`: rappresenta un tracciato articolato caratterizzato da un'elevata densità di curve complesse e strette, utile per sollecitare il tracciamento di traiettoria in spazi ristretti.
- `get_pista_3`: definisce un circuito con curve articolate progettato per prevedere la percorrenza di due giri consecutivi, permettendo di analizzare il comportamento e l'efficacia dei meccanismi di chiusura del ciclo (*loop closure*).
- `get_contapassi`: implementa un percorso speciale caratterizzato da una sequenza ravvicinata di svolte estremamente strette. La sua finalità principale è analizzare le variazioni di stabilità e accuratezza dell'algoritmo di *Pure Pursuit* al variare della distanza di mira (*look-ahead distance*).

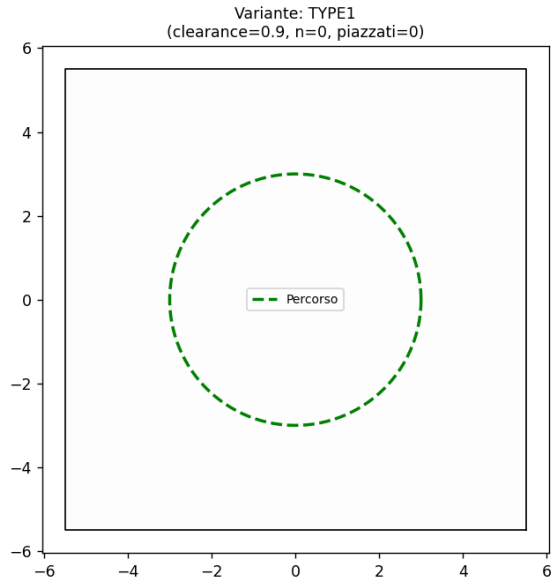
Di seguito sono illustrati i tracciati utilizzati per la conduzione dei test.



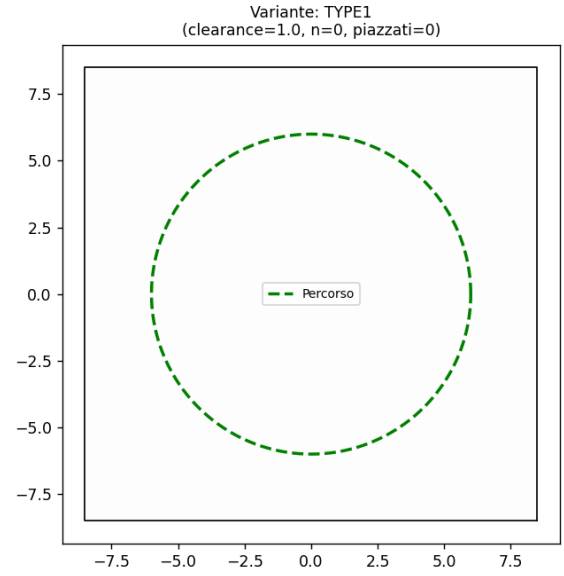
(a) Percorso `straight_short` (5 metri).

(b) Percorso `straight_long` (20 metri).

Figura 3.1: Tracciati rettilinei di prova.

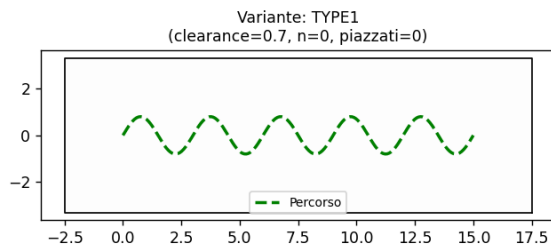


(a) Percorso `circle_medium` ($R = 3$ m).

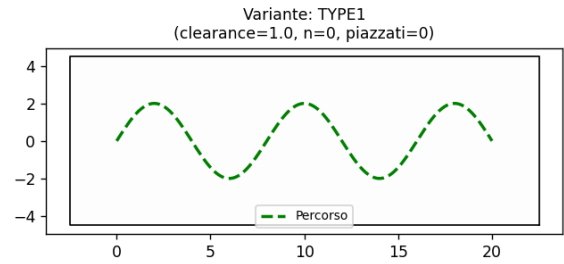


(b) Percorso `circle_large` ($R = 6$ m).

Figura 3.2: Tracciati circolari di prova.

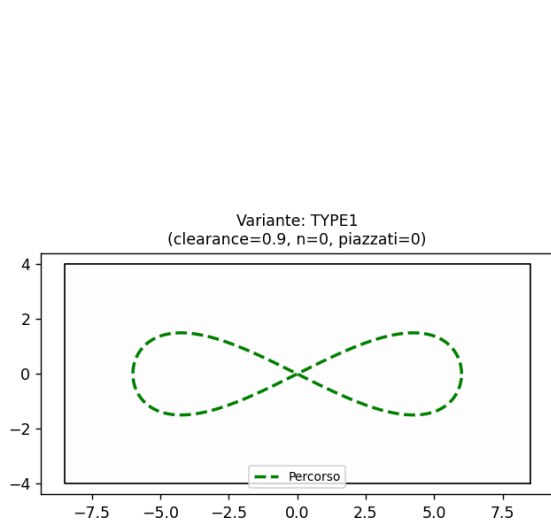


(a) Tracciato `tight_slalom` (15 m, ampiezza 0.8 m, $\lambda = 3$ m).

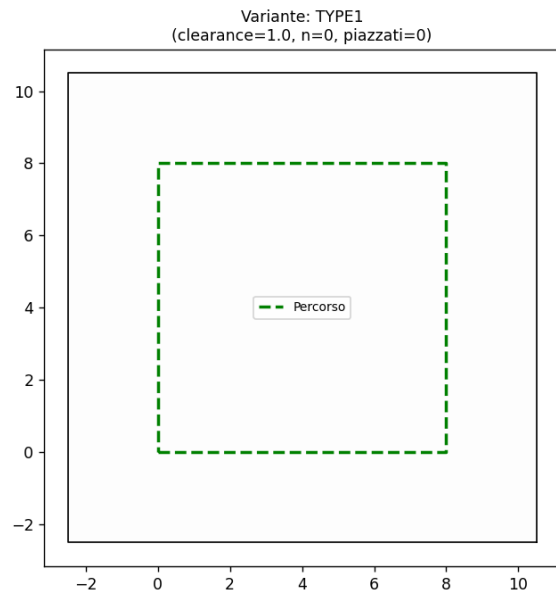


(b) Tracciato `wide_slalom` (lunghezza 20 m, ampiezza 2 m, $\lambda = 8$ m).

Figura 3.3: Confronto tra tracciati di tipo slalom

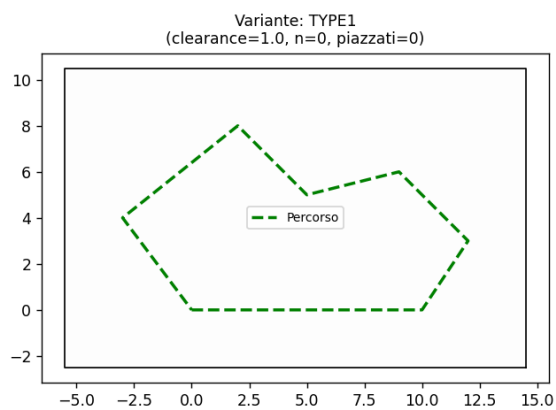


(a) Tracciato a forma di otto (**eight**).

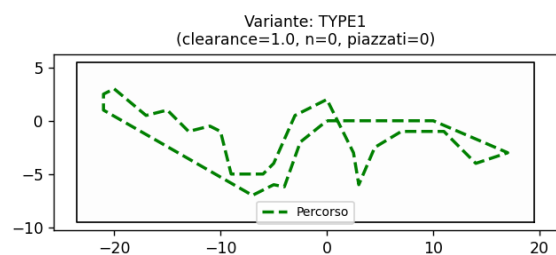


(b) Tracciato a percorso quadrato (**square**).

Figura 3.4: Circuiti di tipo otto e di tipo quadrato

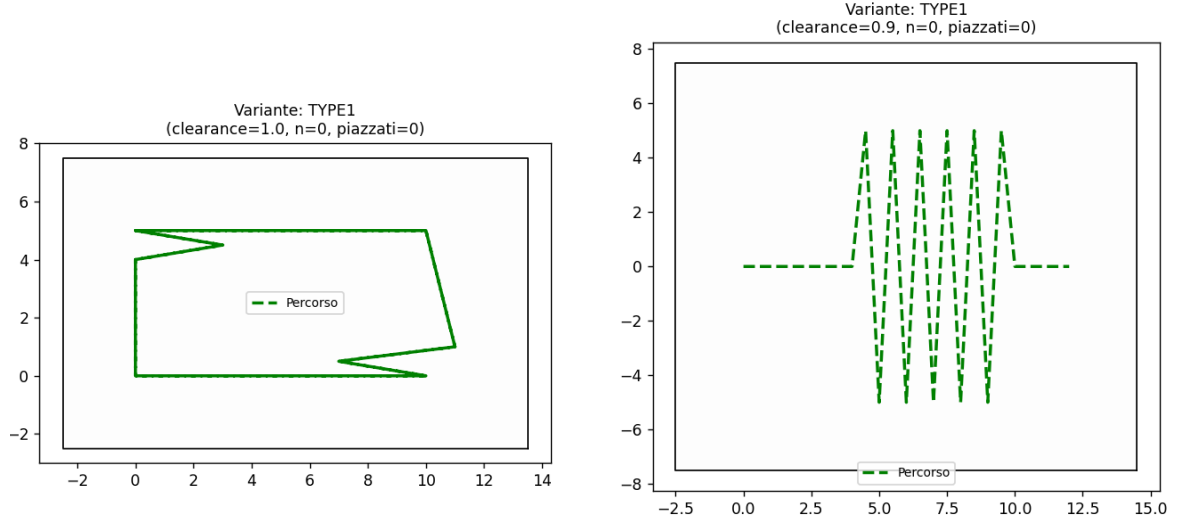


(a) Tracciato del circuito **pista_1**.



(b) Tracciato del circuito **pista_2**.

Figura 3.5: Confronto tra il circuito a geometria semplice (**pista_1**) e quello a geometria complessa (**pista_2**).



(a) Tracciato del circuito `pista_3` su doppio giro per la valutazione della *loop closure*.

(b) Tracciato del circuito `contapassi` per l'analisi al variare della *look-ahead distance*.

Figura 3.6: Tracciati di test: a sinistra la configurazione a doppio giro per la *loop closure* (`pista_3`), a destra il circuito per la valutazione della *look-ahead distance* (`contapassi`).

3.4 Generazione di ambienti predefiniti per i test

3.4.1 Il modulo `environment_presets_pure_pursuit.py`

Il modulo `environment_presets_pure_pursuit.py` consente di generare gli ambienti di test in modo deterministico, garantendone la completa riproducibilità. La creazione e la configurazione degli ambienti avvengono secondo una sequenza ben strutturata. Nello specifico, per ciascun percorso disponibile vengono definite fino a tre varianti (`type1`, `type2` e `type3`), le quali si differenziano sulla base della *clearance* (ossia lo spazio libero da ostacoli da garantire attorno alla traiettoria del robot), del numero complessivo di ostacoli (`n_obstacles`) e del raggio massimo di scansione del sensore LiDAR (`r_max`), così da poterlo impostare al valore più opportuno secondo la tipologia dell'ambiente.

```

1 PRESET_ENV_CONFIG: dict = {
2     "straight_short": {
3         "type1": {"clearance": 0.8, "n_obstacles": 0, "r_max": 4.0},
4         "type2": {"clearance": 0.7, "n_obstacles": 2, "r_max": 8.0},
5         "type3": {"clearance": 0.6, "n_obstacles": 5, "r_max": 4.0},
6     },
7 }

```

Listing 3.3: Esempio di configurazione delle varianti tramite dizionario annidato.

Per quanto riguarda il posizionamento e la configurazione degli ostacoli nell'ambiente di simulazione, il sistema si affida alla **sequenza di Halton** la quale è una sequenza deterministica a *bassa discrepanza* (alta uniformità). Questa caratteristica consente di sparpagliare gli elementi in modo estremamente naturale, omogeneo e uniforme su tutta l'area di gioco, evitando la formazione di cluster o la creazione di semplici corridoi paralleli al percorso.

Dal punto di vista matematico, la sequenza si basa sulla **funzione di riflessione radicale** $\phi_b(n)$, dove b rappresenta un numero intero scelto come base (tipicamente un numero primo). Per ogni indice intero $n \geq 1$, espresso nella sua rappresentazione in base b :

$$n = \sum_{i=0}^k a_i(n) b^i = a_0 + a_1 b + a_2 b^2 + \dots + a_k b^k \quad \text{con } 0 \leq a_i(n) < b \quad (3.2)$$

la funzione di riflessione inverte l'ordine delle cifre rispetto al punto radicale, trasformandole in pesi frazionari:

$$\phi_b(n) = \sum_{i=0}^k a_i(n) b^{-(i+1)} = \frac{a_0}{b} + \frac{a_1}{b^2} + \dots + \frac{a_k}{b^{k+1}} \in [0, 1) \quad (3.3)$$

Per generare le varie proprietà degli ostacoli in uno spazio multidimensionale mantenendole indipendenti tra loro, il sistema associa una base prima distinta p_d a ciascuna dimensione. Il generatore crea così una grande varietà di parametri geometrici, combinando diverse basi:

- **Posizione nello spazio (X, Y) :**
 - **Coordinata X :** Generata tramite la **base 2**.
 - **Coordinata Y :** Generata tramite la **base 3**.
- **Dimensione e Raggio:** Determinata tramite la **base 5** (mappata nell'intervallo tra dimensione minima e massima).
- **Geometria dell'ostacolo:** Selezionata tramite la **base 7** (suddivisa in tre intervalli per scegliere tra cerchio, rettangolo o poligono).
- **Proprietà specifiche della forma:**
 - **Proporzioni del rettangolo (*aspect ratio*):** Generate tramite la **base 11**.
 - **Rotazione del poligono:** Generata tramite la **base 13**.
 - **Numero di lati del poligono:** Calcolato in modo deterministico tramite aritmetica modulare dall'indice dell'iterazione ($idx \pmod 3$), alternando 3, 5 e 7 lati) senza fare uso di una base di Halton.

Infine, per evitare che i primi punti generati si concentrino nell'origine dell'ambiente, viene applicato un offset iniziale all'indice n ($n = n_0 + \text{attempt}$, con $n_0 = 17$).

Nel seguente codice viene mostrata la procedura di generazione deterministica degli ostacoli basata sulle sequenze di Halton.

```
1 idx = halton_offset + attempt
2 attempt += 1
3
4 fx = _halton(idx, 2)
5 fy = _halton(idx, 3)
6 cx = b_left + fx * span_x
7 cy = b_bottom + fy * span_y
8
9 # Dimensione e forma variate deterministicamente da altre due
10 # dimensioni della sequenza di Halton (basi 5 e 7), indipendenti
11 # dalla posizione, per avere varieta di feature per l'ICP.
12 size_frac = _halton(idx, 5)
13 r = min_size + size_frac * (max_size - min_size)
14
15 shape_frac = _halton(idx, 7)
16 if shape_frac < 0.34:
17     shape_kind = "circle"
18 elif shape_frac < 0.67:
19     shape_kind = "rectangle"
20 else:
21     shape_kind = "polygon"
22
23 if shape_kind == "circle":
24     candidate = Point(cx, cy).buffer(r, resolution=24)
25     if _is_safe(env, candidate, path_line, clearance):
26         env.add_circle(cx, cy, r)
27         placed += 1
28
29 elif shape_kind == "rectangle":
30     # Rettangolo non quadrato (rapporto d'aspetto variato) per
31     # aggiungere ulteriore diversita geometrica
32     aspect = 0.6 + 0.8 * _halton(idx, 11) # 0.6 - 1.4
33     w, h = r * 2.0, r * 2.0 * aspect
34     xmin_r, ymin_r = cx - w / 2, cy - h / 2
35     xmax_r, ymax_r = cx + w / 2, cy + h / 2
36     candidate = box(xmin_r, ymin_r, xmax_r, ymax_r)
37     if _is_safe(env, candidate, path_line, clearance):
```

```

38     env.add_rectangle(xmin_r, ymin_r, xmax_r, ymax_r)
39     placed += 1
40
41 else: # poligono regolare (triangolo, pentagono o esagono)
42     n_sides = 3 + (int(idx) % 3) * 2 # alterna 3, 5, 7 lati
43     rotation = 2 * np.pi * _halton(idx, 13)
44     verts, candidate = _make_polygon_geom(cx, cy, r, n_sides, rotation)
45     if _is_safe(env, candidate, path_line, clearance):
46         env.add_polygon(verts)
47         placed += 1

```

Listing 3.4: Posizionamento deterministico e diversificazione geometrica degli ostacoli tramite sequenza di Halton.

Esempio di Calcolo della Sequenza

Per comprendere concretamente la determinazione delle coordinate di un candidato, consideriamo il primo tentativo di generazione ($\text{attempt} = 1$) con offset $n_0 = 17$, corrispondente all'indice $n = 18$.

1. **Coordinata X (Base $b = 2$):** L'indice 18 convertito in base 2 è $(10010)_2$, ovvero:

$$18 = 0 \cdot 2^0 + 1 \cdot 2^1 + 0 \cdot 2^2 + 0 \cdot 2^3 + 1 \cdot 2^4 \quad (3.4)$$

Applicando la riflessione radicale $\phi_2(18)$, le cifre vengono invertite nei pesi frazionari:

$$\phi_2(18) = \frac{0}{2^1} + \frac{1}{2^2} + \frac{0}{2^3} + \frac{0}{2^4} + \frac{1}{2^5} = \frac{1}{4} + \frac{1}{32} = 0.28125 \quad (3.5)$$

2. **Coordinata Y (Base $b = 3$):** L'indice 18 convertito in base 3 è $(200)_3$, ovvero:

$$18 = 0 \cdot 3^0 + 0 \cdot 3^1 + 2 \cdot 3^2 \quad (3.6)$$

Invertendo le cifre per la riflessione radicale $\phi_3(18)$:

$$\phi_3(18) = \frac{0}{3^1} + \frac{0}{3^2} + \frac{2}{3^3} = \frac{2}{27} \approx 0.07407 \quad (3.7)$$

Il primo punto candidato generato nello spazio unitario $[0, 1]^2$ sarà quindi la coppia $(\phi_2(18), \phi_3(18)) = (0.28125, 0.07407)$, la quale verrà successivamente scalata sulle dimensioni reali $(\Delta x, \Delta y)$ dei confini della mappa.

Nel seguente codice viene mostrato come esso viene calcolato

```

1 def _halton(index: int, base: int) -> float:
2     # Variabile accumulatore per il risultato finale (inizialmente 0.0)

```

```

3   result = 0.0
4   # Peso della cifra corrente: parte come (1 / base) e viene diviso ad
   ogni ciclo
5   f = 1.0 / base
6   # Copia locale dell'indice da scomporre
7   i = index
8   # Ciclo che "riflette" le cifre dell'indice convertito nella base scelta
9   while i > 0:
10      # 1. (i % base) estrae l'ultima cifra di 'i' nella base 'base'
11      # 2. Moltiplica la cifra per il peso corrente 'f' e la somma al
12      risultato
13      result += f * (i % base)
14      # Riduce 'i' eliminando l'ultima cifra appena elaborata
15       #(divisione intera)
16      i //= base
17      # Riduce il peso per la cifra successiva
18      f /= base
19
20  return result

```

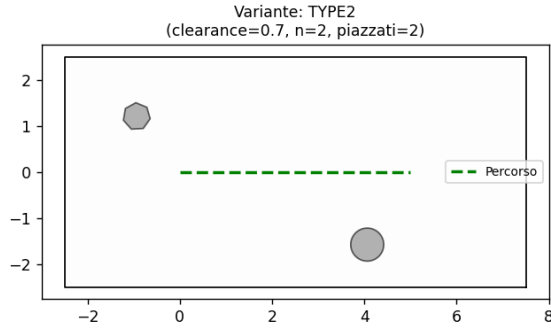
Listing 3.5: Implementazione della sequenza a bassa discrepanza di Halton per la generazione deterministica.

Prima che un ostacolo candidato venga confermato ed inserito definitivamente nella mappa, il sistema effettua attraverso la funzione `_is_safe` un rigoroso controllo di sicurezza verificando tre condizioni fondamentali:

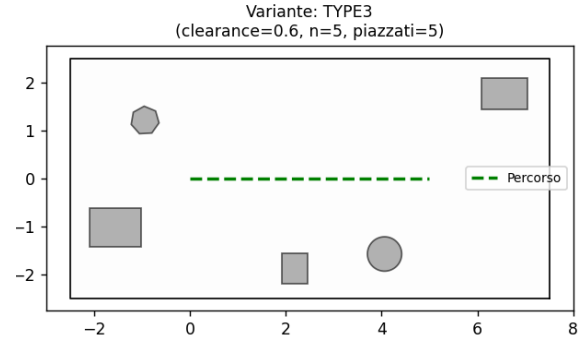
1. Il rispetto della distanza minima di sicurezza (*clearance*) dal tracciato di riferimento del robot.
2. L'assenza di sovrapposizioni o intersezioni con altri ostacoli già confermati nell'ambiente.
3. La completa inclusione dell'ostacolo all'interno dei confini (*bounds*) della mappa.

Per ottimizzare le prestazioni e ridurre i tempi di calcolo, viene adottato un meccanismo di caching: ogni ambiente viene generato matematicamente solo alla prima richiesta, per poi essere salvato in memoria e restituito all'istante nelle chiamate successive. Infine, il modulo mette a disposizione la funzione `plot_all_environments`, uno strumento grafico interattivo che permette di navigare tra i vari tracciati e confrontare visivamente, su una griglia, le tre varianti di difficoltà previste per ciascuno di essi.

Di seguito vengono mostrati alcuni esempi di ambienti generati per le diverse tipologie di test.

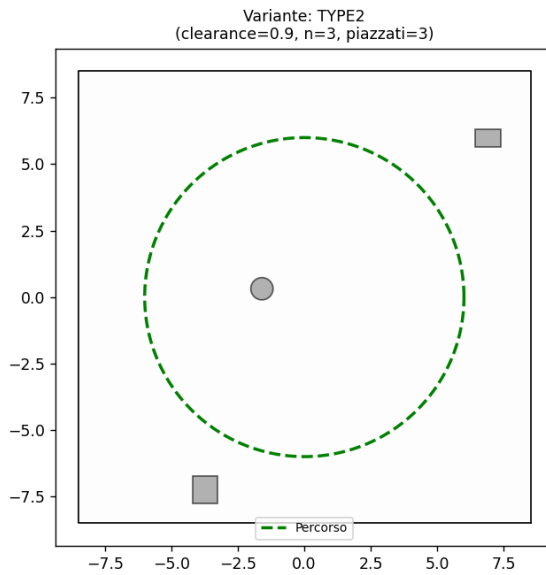


(a) Variante **type2** (2 ostacoli).

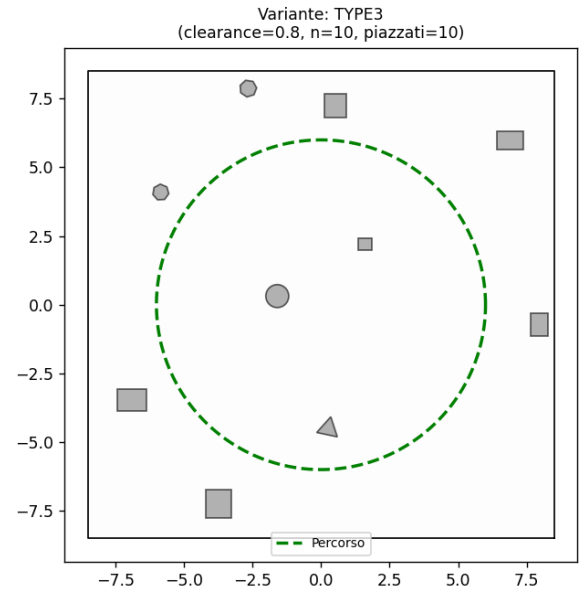


(b) Variante **type3** (5 ostacoli).

Figura 3.7: Confronto del tracciato rettilineo breve (**straight_short**): la variante **type2** presenta un numero ridotto di ostacoli rispetto alla variante **type3**, che mostra un ambiente a maggiore densità ed elementi geometrici più ravvicinati.



(a) Variante **type2** (3 ostacoli).



(b) Variante **type3** (10 ostacoli).

Figura 3.8: Confronto del tracciato circolare ampio (**circle_large**): la variante **type2** presenta una minore densità di ostacoli nell'area di navigazione rispetto alla variante **type3**, caratterizzata da una presenza più elevata di elementi ostacolanti distribuiti nell'ambiente.

Si ricorda che la variante `type1` rappresenta l'ambiente privo di ostacoli già mostrata precedentemente, utilizzato per analizzare il comportamento del robot quando l'inseguimento della traiettoria si affida esclusivamente all'odometria. Quest'ultima può essere sia **ideale** (in assenza di rumore, per cui la stima odometrica coincide costantemente con la posizione reale del robot, se esso viene inizializzato nella posizione corretta), sia **reale** (soggetta a rumore e simulata tramite il modulo `noisy_odometry.py`, in cui gli errori accumulati deviano la stima dalla posizione effettiva).

In quest'ultimo scenario, il robot deve fare affidamento sul sensore LiDAR per stimare correttamente la propria posa. Risulta quindi fondamentale valutare le prestazioni del sistema in ambienti caratterizzati da una diversa densità di ostacoli, come avviene nelle configurazioni `type2` e `type3`.

3.5 Simulazione dell'Odometria Rumorosa

3.5.1 Il modulo `noisy_odometry.py`

Il modulo `noisy_odometry.py` consente di simulare un'odometria realistica caratterizzata da componenti di errore sia continue che intermittenti. Il modello cinematico della posa $[x, y, \theta]^T$ integra due livelli di disturbo sui comandi di velocità lineare v e angolare ω :

1. **Rumore di fondo continuo:** Un disturbo gaussiano a media zero le cui deviazioni standard σ_v e σ_ω sono proporzionali alle velocità istantanee:

$$\sigma_v = k_{\text{base}} \cdot |v|, \quad \sigma_\omega = k_{\text{base}} \cdot |\omega| \quad (3.8)$$

con $k_{\text{base}} = 0.01$. Questo termine modella le piccole imprecisioni sistematiche e il rumore fisiologico di rotolamento.

Dati v e ω come i valori senza rumore dal punto di vista dell'implementazione, il calcolo e l'applicazione del rumore di fondo avvengono tramite il seguente codice:

```

1 std_v = self.base_noise * abs(v)
2 std_w = self.base_noise * abs(omega)
3
4 noisy_v = v + np.random.normal(0, std_v) if std_v > 0 else v
5 noisy_omega = omega + np.random.normal(0, std_w) if std_w > 0 else
6     omega

```

2. **Disturbi sporadici (slittamenti):** Ad ogni istante temporale dt , con una probabilità $p_{\text{slip}} = 0.03$ (3%), viene iniettato un errore improvviso selezionando casualmente tra un disturbo lineare, angolare o combinato:

- *Errore lineare:* $\Delta v \sim \mathcal{U}(-0.3, 0.3)$ m/s;
- *Errore angolare:* $\Delta \omega \sim \mathcal{U}(-0.5, 0.5)$ rad/s.

L'iniezione dei disturbi sporadici viene gestita dal seguente blocco di codice:

```

1 if np.random.random() < self.slip_prob:
2     # Sceglie a caso il tipo di disturbo: lineare, angolare o entrambi
3     disturbo = np.random.choice(['lineare', 'angolare', 'entrambi'])
4
5     if disturbo in ['lineare', 'entrambi']:
6         # Aggiunge un picco casuale uniforme alla velocita
7         noisy_v += np.random.uniform(-self.slip_v_mag, self.slip_v_mag)
8
9     if disturbo in ['angolare', 'entrambi']:
10        # Aggiunge uno strattone improvviso alla rotazione
11        noisy_omega += np.random.uniform(-self.slip_w_mag, self.
slip_w_mag)

```

Integrazione della Posa e Normalizzazione

Una volta calcolati i valori di velocità eventualmente alterati dal rumore, il nuovo stato del robot $[x, y, \theta]^T$ viene aggiornato tramite un'integrazione numerica di Eulero di primo ordine. Successivamente, l'orientamento θ viene riproiettato nell'intervallo $[-\pi, \pi]$ per garantire la continuità angolare.

Di seguito viene riportata l'implementazione dell'integrazione e della normalizzazione dell'angolo:

```

1 # Integrazione di Eulero con i comandi alterati
2 self.state[0] += noisy_v * np.cos(self.state[2]) * dt
3 self.state[1] += noisy_v * np.sin(self.state[2]) * dt
4 self.state[2] += noisy_omega * dt
5
6 # Normalizzazione dell'angolo nell'intervallo [-pi, pi]
7 self.state[2] = (self.state[2] + np.pi) % (2 * np.pi) - np.pi
8
9 return self.state.copy()

```

3.6 Esecuzione degli esperimenti, visualizzazione e salvataggio dei risultati

3.6.1 Il modulo `main_pure_pursuit.py`

Come spiegato in precedenza, il modulo `main_pure_pursuit.py` costituisce il punto di ingresso principale dell'applicazione.

In primo luogo, l'applicazione recupera l'elenco dei tracciati da testare mediante la chiamata:

```
1 path_names = PrefabricatedPaths.list_presets()
```

Successivamente, vengono definiti i tipi di ambiente da valutare e le distanze di *look-ahead* da esaminare, specificandoli rispettivamente all'interno degli array `variants` e `lookahead_distances`.

Nel ciclo principale, il sistema esegue ciascun test invocando la funzione `run_simulation` (definita nel modulo `pure_pursuit_simulation.py`). Ogni scenario viene valutato secondo quattro combinazioni operative distinte, ottenute alternando l'impiego di un'odometria ideale rispetto a una rumorosa, unitamente alla presenza o assenza del meccanismo di *loop closure*.

Al termine di ogni esperimento, viene richiamata la funzione `export_group_plots` (definita in `visualizer_pure_pursuit.py`) per salvare i grafici generati all'interno della cartella di destinazione `img_pure_pursuit` (creata automaticamente qualora non fosse già presente). Infine, viene istanziata la classe `InteractiveVisualizer`, anch'essa definita in `visualizer_pure_pursuit.py`, che fornisce un'interfaccia grafica interattiva per l'ispezione e l'analisi dei risultati ottenuti.

3.6.2 Il modulo `visualizer_pure_pursuit.py`

Come accennato in precedenza, il modulo viene richiamato dal programma principale `main_pure_pursuit.py` per fornire un'interfaccia grafica avanzata e interattiva. Questa permette di visualizzare i risultati dell'esecuzione dell'algoritmo Pure Pursuit nei diversi scenari, mostrando l'errore medio e la somma degli errori calcolati per ciascun esperimento, unitamente ai relativi grafici.

La gestione della visualizzazione è affidata alla classe `InteractiveVisualizer`, la quale sfrutta la funzione `draw_robot` (definita nel modulo `visualizer.py`) per disegnare il robot quando andiamo ad osservare la simulazione animata del test, offrendo all'utente la possibilità di mettere in pausa la simulazione o di avanzare fotogramma per fotogramma (*step-by-step*).

I valori di errore vengono calcolati da una funzione dedicata presente all'interno dello stesso modulo, la funzione `calc_errors`. Essa determina la distanza minima dal

percorso di riferimento per ciascun punto della traiettoria reale e di quella stimata, così da permettere il calcolo dell'errore medio.

Infine, la gestione e l'esportazione dei grafici su disco sono garantite da due ulteriori funzioni dello stesso modulo: `get_unique_filepath` genera un percorso file univoco per evitare la sovrascrittura di dati preesistenti appendendo un suffisso numerico progressivo del tipo `_n` (con $n \in \mathbb{N}$), mentre `export_group_plots` provvede ad archiviare i grafici (all'interno di una cartella che viene eventualmente generata se assente chiamata `img_pure_pursuit`) man mano che vengono generati, prevenendo la saturazione della memoria ed evitando il classico errore di sistema `'Fail to allocate bitmap'`.

Capitolo 4

Algoritmo Pure Pursuit ed esecuzione di esso

4.1 Introduzione di capitolo

In questo capitolo viene descritto nel dettaglio il funzionamento dell'algoritmo *Pure Pursuit* e la relativa architettura di simulazione. Nello specifico, verranno analizzati i moduli `pure_pursuit.py`, contenente la logica dell'algoritmo di inseguimento, e `pure_pursuit_simulation.py`, responsabile della gestione e del coordinamento della simulazione dinamica del robot.

4.2 L'algoritmo Pure Pursuit

L'algoritmo *Pure Pursuit* (Inseguimento Puro) è un metodo di controllo cinematico progettato per guidare un robot lungo una traiettoria prestabilita. L'implementazione calcola in tempo reale i comandi di velocità per orientare il veicolo verso un punto di mira dinamico, denominato *look-ahead point*.

Dal punto di vista geometrico, il funzionamento dell'algoritmo offre un'interpretazione molto intuitiva: attorno alla posizione attualmente stimata dai sensori del robot viene tracciata una circonferenza di raggio pari alla distanza di *look-ahead* (L_d), centrata sulla posa stimata del veicolo. Il punto di mira viene individuato determinando l'intersezione tra questa circonferenza e il percorso di riferimento che si desidera inseguire. Man mano che il robot avanza, la circonferenza si sposta con esso, facendo scorrere il punto di intersezione lungo la traiettoria.

In sintesi, il processo si articola in due passaggi operativi fondamentali. In primo luogo, si procede all'identificazione del punto di mira ricercando l'intersezione tra la circonferenza di raggio L_d e la traiettoria pianificata, selezionando il segmento opportuno più avanzato rispetto al movimento del robot. Successivamente, si passa al calcolo della

manovra di sterzata attraverso la determinazione della curvatura dell'arco di cerchio che connette la posa attualmente stimata del veicolo al *look-ahead point*, dalla quale vengono infine ricavati i comandi di velocità angolare necessari per il controllo.

4.3 Il modulo `pure_pursuit.py`

Nel modulo `pure_pursuit.py` l'algoritmo di pure pursuit viene implementato dalla classe `PurePursuitController` di seguito vengono esposti i vari metodi della classe.

Inizializzazione del Controllore (`__init__`)

Il metodo `__init__` si occupa di configurare lo stato iniziale e i parametri operativi della classe `PurePursuitController`. La firma del metodo e i suoi valori di default sono definiti come segue:

```
1 def __init__(
2     self,
3     lookahead_distance: float = 0.5,
4     max_angular_velocity: float = 2.0,
5     target_linear_velocity: float = 0.4,
6     stop_tolerance: float = 0.1,
7     search_window: int = 50
8 ):

```

Nello specifico, il metodo definisce ed inizializza diverse variabili di istanza fondamentale per il funzionamento del controllore.

Per quanto riguarda i parametri cinematici e geometrici, la variabile `self.L_d` imposta la distanza di *look-ahead* ($L_d = 0.5$ m), ovvero il raggio della circonferenza centrata sulla posa stimata del robot, utilizzata per intercettare il punto di mira (*goal point*) lungo il tracciato. La velocità lineare di avanzamento costante desiderata per il veicolo è definita da `self.target_v` ($v_{\text{target}} = 0.4$ m/s), mentre `self.max_omega` ($\omega_{\text{max}} = 2.0$ rad/s) impone un vincolo di saturazione sulla velocità angolare, limitando la massima rotazione ammissibile per garantire la stabilità dinamica del robot.

La gestione del progresso e la delimitazione dell'area di calcolo sono affidate a due variabili chiave. L'indice interno `self._last_idx`, inizializzato a 0, tiene traccia dell'ultimo *waypoint* del percorso validato; esso impedisce al controllore di valutare segmenti già superati, garantendo un avanzamento unidirezionale lungo la traiettoria. Parallelamente, `self.search_window` ($W_s = 50$) stabilisce l'ampiezza della finestra di ricerca locale, definendo il numero massimo di *waypoint* successivi a `_last_idx` da scansionare per individuare l'intersezione con il cerchio di *look-ahead*. Tale strategia evita di scorrere inutilmente l'intero array del percorso ad ogni ciclo di controllo, riducendo

significativamente la complessità computazionale. Inoltre, questo approccio previene il problema dell'arresto prematuro nei tracciati chiusi o in quelli in cui il punto di arrivo si trova fisicamente vicino al punto di partenza ma deve essere raggiunto percorrendo un tragitto più lungo. Senza questo accorgimento, il robot potrebbe dirigersi erroneamente verso il traguardo prima del tempo.

Infine, la condizione di arresto è regolata dalla variabile `self.stop_tolerance` ($d_{\text{stop}} = 0.1\text{ m}$), la quale rappresenta la soglia di tolleranza sulla distanza euclidea residua rispetto all'ultimo punto della traiettoria. Questa coordinata, valutata insieme al parametro `max_index_gap` (il quale ha un valore fissato a 50), determina il momento preciso in cui il robot si considera giunto a destinazione e può arrestare la propria corsa.

Ripristino dello Stato del Controllore (`reset`)

Il metodo `reset` è una funzione di utilità dedicata alla gestione dello stato interno del controllore. La sua definizione e implementazione sono riportate di seguito:

```

1 def reset(self) -> None:
2     """
3     Resetta il progresso lungo il path (da chiamare se si cambia percorso).
4     """
5     self._last_idx = 0

```

Questo metodo di utilità viene invocato per azzerare la variabile di istanza `self._last_idx`, ripristinandone il valore iniziale pari a 0.

Come illustrato in precedenza, la variabile `_last_idx` viene impiegata dal controllore come stato interno per memorizzare il progresso del robot lungo la traiettoria, evitando in tal modo che vengano presi in considerazione *waypoint* già superati. Di conseguenza, il reset di tale variabile risulta fondamentale prima dell'avvio di ogni nuova simulazione su un tracciato differente, garantendo il corretto ripristino delle condizioni iniziali della struttura dati.

Calcolo del Punto di Mira (`find_lookahead_point`)

Il metodo `find_lookahead_point` rappresenta il cuore geometrico dell'algoritmo *Pure Pursuit*. Il suo compito è determinare le coordinate esatte del punto di *look-ahead* sulla traiettoria desiderata, calcolando l'intersezione tra i segmenti che compongono il percorso e una circonferenza di raggio L_d centrata sul robot.

```

1 def find_lookahead_point(
2     self, robot_pose: np.ndarray, path: np.ndarray
3 ) -> np.ndarray:

```

L'elaborazione si articola in diverse fasi logiche e matematiche sequenziali.

Nella prima fase si verifica se la distanza tra il robot e il punto finale è minore o uguale al raggio di look-ahead (L_d) e se il robot si trova in prossimità della fine del tracciato (entro il limite stabilito da `search_window`). In caso affermativo, l'algoritmo seleziona direttamente il traguardo come punto di mira, evitando l'esecuzione della successiva ricerca del waypoint.

Questa verifica preliminare viene introdotta per risolvere un'anomalia di comportamento in cui il robot, giunto in prossimità del punto d'arrivo, tende a scartare di lato ed eseguire una traiettoria circolare di fianco al target prima di riuscire ad arrestarsi o terminare la simulazione.

Tale comportamento è direttamente imputabile all'interazione tra il raggio di *look-ahead* e la tolleranza di arresto del sistema (*stop tolerance*), impostata nel nostro caso a 0.1 m. Quando il valore di L_d è significativamente maggiore della tolleranza di arresto, il cerchio di look-ahead interseca e supera il punto finale del percorso prima che il centro del robot sia entrato nella zona di arresto. A quel punto, poiché l'unico punto visibile del tracciato risulta situato alle spalle o lateralmente al veicolo, l'algoritmo Pure Pursuit genera continui comandi di sterzata per riallinearsi, innescando una traiettoria circolare a fianco del target.

```

1 final_point = path[-1, :2]
2 dist_to_final = np.linalg.norm(final_point - current_pos)
3 index_distance_to_end = (n - 1) - self._last_idx
4
5 if dist_to_final <= L_d and index_distance_to_end <= self.search_window:
6     self._last_idx = n - 1
7     return final_point.copy()

```

Listing 4.1: Controlli per il puntamento diretto del traguardo.

Successivamente, si procede con la ricerca effettiva del punto di mira. Per ridurre l'onere computazionale derivante dalla scansione dell'intero tracciato a ogni iterazione di controllo, il metodo limita la ricerca all'interno di una finestra locale (`search_window`).

L'esplorazione dei segmenti del percorso parte dall'ultimo indice validato (`self._last_idx`) e si estende al massimo per un numero di *waypoint* pari a `self.search_window`. L'uso di `self._last_idx` garantisce che il robot non prenda in considerazione segmenti già percorsi, mentre la *search window* impedisce di selezionare punti troppo lontani lungo il tracciato, evitando il taglio delle curve e l'imbocco di scorciatoie indebite. L'insieme di queste contromisure previene il problema della terminazione prematura precedentemente descritto — la cui verifica finale resta affidata all'apposito controllo di terminazione — e preserva un'elevata efficienza computazionale.

Per ogni segmento analizzato, definito dai punti $P_1 = (x_1, y_1)$ e $P_2 = (x_2, y_2)$, si procede con una semplificazione del sistema di riferimento: le coordinate globali vengo-

no traslate affinché la posizione stimata corrente del robot coincida temporaneamente con l'origine degli assi $(0,0)$.

La ricerca dell'intersezione si basa quindi sulla risoluzione algebrica del sistema tra l'equazione della circonferenza di raggio L_d e la retta passante per il segmento traslato. Siano $dx = x_2 - x_1$ e $dy = y_2 - y_1$ le componenti direzionali del segmento, si calcolano il quadrato della lunghezza del segmento dr^2 e il determinante della matrice delle coordinate D :

$$dr^2 = dx^2 + dy^2$$

$$D = x_1 \cdot y_2 - x_2 \cdot y_1$$

Da questi parametri si valuta il discriminante Δ , il quale determina la natura dell'intersezione geometrica:

$$\Delta = L_d^2 \cdot dr^2 - D^2$$

Nel caso in cui $\Delta < 0$, la retta risulta esterna alla circonferenza (assenza di intersezioni) e l'algoritmo passa direttamente all'analisi del segmento successivo. Se invece $\Delta \geq 0$, la retta è secante o tangente, e le formule algebriche restituiscono fino a due possibili soluzioni per le coordinate di intersezione (x, y) .

Poiché le equazioni analitiche calcolano le intersezioni rispetto ad una retta di lunghezza infinita, si rende necessaria una validazione dei confini: l'algoritmo verifica rigorosamente che i punti trovati giacciono fisicamente all'interno della porzione limitata del segmento, confrontando le coordinate calcolate con i valori minimi e massimi delle componenti X e Y dei punti P_1 e P_2 , introducendo un margine di tolleranza $\epsilon = 10^{-9}$.

A questo punto si passa alla selezione del candidato ottimale: nel caso in cui vi siano due intersezioni valide sullo stesso segmento, l'algoritmo seleziona il punto candidato più vicino al nodo terminale del segmento (P_2), garantendo il naturale progresso lungo il percorso. Successivamente, viene verificato che tale candidato si trovi effettivamente più avanti rispetto alla posizione corrente del veicolo; se la condizione è soddisfatta, l'indice `_last_idx` viene aggiornato e il ciclo di ricerca si interrompe.

Infine, qualora al termine dell'esplorazione della finestra di ricerca non venga individuata alcuna intersezione geometrica valida, il sistema attua un comportamento di *fallback* a scopo di sicurezza. In tal caso, viene restituito l'ultimo *waypoint* visitato valido, evitando l'interruzione imprevista del controllore e consentendo al robot di proseguire l'inseguimento.

Di seguito si riporta un frammento del metodo.

```

1 for i in range(start_index, end_index):
2     # Trasla il segmento di percorso ponendo il robot all'origine (0,0) per
    # facilitare i calcoli
3     p1 = path[i, :2] - current_pos

```

```

4     p2 = path[i + 1, :2] - current_pos
5     # intersezione Retta-Circonferenza
6     x1, y1 = p1
7     x2, y2 = p2
8     dx = x2 - x1
9     dy = y2 - y1
10    dr2 = dx * dx + dy * dy
11    # Se i due punti del segmento coincidono (distanza zero), salta al
prossimo
12    if dr2 < 1e-12:
13        continue
14    # Applica la formula matematica dell'intersezione retta-circonferenza
15    D = x1 * y2 - x2 * y1
16    discriminant = (L_d ** 2) * dr2 - D ** 2
17
18    if discriminant < 0:
19        continue
20
21    # Se c'e' intersezione, calcola le possibili soluzioni (fino a due punti
di intersezione)
22    sqrt_disc = np.sqrt(discriminant)
23    sgn_dy = 1.0 if dy >= 0 else -1.0
24
25    # Soluzioni per la prima intersezione
26    sol_x1 = (D * dy + sgn_dy * dx * sqrt_disc) / dr2
27    sol_y1 = (-D * dx + abs(dy) * sqrt_disc) / dr2
28    # Soluzioni per la seconda intersezione
29    sol_x2 = (D * dy - sgn_dy * dx * sqrt_disc) / dr2
30    sol_y2 = (-D * dx - abs(dy) * sqrt_disc) / dr2
31    # Ri-trasla le soluzioni nelle coordinate globali originali
32    sol_pt1 = np.array([sol_x1, sol_y1]) + current_pos
33    sol_pt2 = np.array([sol_x2, sol_y2]) + current_pos
34
35    #... controllo se le intersezioni sono contenute nel segmento...
36
37    # Se entrambe le intersezioni sono valide, sceglie quella piu' vicina al
punto finale
38    # del segmento (per procedere in avanti)
39    next_pt = path[i + 1, :2]
40    if pt1_valid and pt2_valid:
41        if np.linalg.norm(sol_pt1 - next_pt) < np.linalg.norm(sol_pt2 -

```

```

next_pt):
42     candidate = sol_pt1
43     else:
44         candidate = sol_pt2
45     elif pt1_valid:
46         candidate = sol_pt1
47     else:
48         candidate = sol_pt2
49
50     # Assicura che il punto scelto sia effettivamente piu' avanti rispetto
51     # alla posizione attuale next_pt è la fine del segmento candidate
52     # deve essere più vicino a next_pt
53     if np.linalg.norm(candidate - next_pt) <= np.linalg.norm(current_pos -
next_pt):
54         goal_pt = candidate
55         # Aggiorna l'indice così' al prossimo ciclo partira' da qui
56         self._last_idx = i
57         break
58     else:
59         self._last_idx = i + 1
60
61     # Fallback: se nessuna intersezione viene trovata, punta semplicemente all'
    ultimo punto visitato valido
62     if goal_pt is None:
63         fallback_idx = min(self._last_idx, n - 1)
64         goal_pt = path[fallback_idx, :2].copy()
65
66     return goal_pt

```

Calcolo dei Comandi Cinematici basato sull'Angolo di Errore (compute_commands)

Il metodo `compute_commands` implementa la legge di controllo del *Pure Pursuit* basata sulla formulazione trigonometrica dell'angolo di errore α . A partire dalla posa attuale stimata del robot e dalla traiettoria di riferimento, la funzione genera la coppia di comandi di velocità lineare e angolare (v, ω) , nel nostro caso consideriamo v come costante.

```

1 def compute_commands(
2     self, robot_pose: np.ndarray, path: np.ndarray
3 ) -> tuple[float, float]:

```

L'algoritmo si sviluppa attraverso una sequenza definita di fasi logiche e matematiche. Inizialmente, prima di eseguire qualsiasi calcolo geometrico, vengono valutate le condizioni di arresto misurando la distanza euclidea residua d_{goal} tra la posizione stimata corrente del robot (x_r, y_r) e il traguardo finale $(x_{\text{final}}, y_{\text{final}})$:

$$d_{\text{goal}} = \sqrt{(x_{\text{final}} - x_r)^2 + (y_{\text{final}} - y_r)^2}$$

Se il valore di d_{goal} scende al di sotto della soglia di tolleranza impostata (`self.stop_tolerance`) e contestualmente viene verificata la prossimità logica al termine dell'array (ovvero un divario contenuto entro un margine definito da `max_index_gap` punti), il controllore arresta il veicolo restituendo velocità nulle (0.0, 0.0).

Successivamente si passa alla determinazione del vettore target e della distanza l_d . Invocando il metodo `find_lookahead_point`, la funzione individua le coordinate del punto di mira globale $(x_{\text{target}}, y_{\text{target}})$. Da questo si ricavano le componenti cartesiane del vettore posizione $dx = x_{\text{target}} - x_r$ e $dy = y_{\text{target}} - y_r$, unitamente alla distanza di *look-ahead* effettiva:

$$l_d = \sqrt{dx^2 + dy^2}$$

In questa fase si assicura inoltre che l_d mantenga un valore minimo soglia ($l_d \geq 0.001$ m) per prevenire singolarità matematiche e divisioni per zero nei calcoli successivi.

La fase seguente riguarda il calcolo dell'angolo di errore α . L'orientamento assoluto verso il punto di mira viene determinato attraverso la funzione arcotangente a due argomenti:

$$\theta_{\text{target}} = \text{atan2}(dy, dx)$$

Da questo valore si ottiene l'errore di orientamento α , ossia l'angolo formato tra l'asse longitudinale del robot e il vettore che punta al target. Tale angolo viene calcolato per differenza e opportunamente normalizzato nell'intervallo $[-\pi, \pi]$ mediante la relazione:

$$\alpha = \text{atan2}(\sin(\theta_{\text{target}} - \theta_r), \cos(\theta_{\text{target}} - \theta_r))$$

A questo punto il controllore gestisce l'eventuale presenza di un target arretrato ($|\alpha| > \frac{\pi}{2}$). Qualora il valore assoluto dell'angolo α superi i 90° , il punto di mira si trova nell'emisfero posteriore del veicolo. Per consentire un rapido riallineamento del robot, il sistema impone la velocità lineare di crociera v_{target} e applica la massima velocità angolare consentita nella direzione dell'errore:

$$\omega = \text{sgn}(\alpha) \cdot \omega_{\text{max}}$$

Infine, se il punto si trova correttamente di fronte al veicolo ($|\alpha| \leq \frac{\pi}{2}$), si procede con il calcolo della curvatura e la generazione dei comandi. In questa condizione si applica

l'equazione trigonometrica fondamentale del *Pure Pursuit* per ricavare la curvatura k dell'arco di circonferenza necessario al raddrizzamento:

$$k = \frac{2 \sin(\alpha)}{l_d}$$

Mantenendo costante la velocità lineare ($v = v_{\text{target}}$), la velocità angolare desiderata viene ricavata dalla relazione $\omega = v \cdot k$ e infine vincolata tramite una funzione di saturazione (*clipping*), garantendo che il valore rientri nei limiti meccanici ed operativi del sistema ($\omega \in [-\omega_{\text{max}}, \omega_{\text{max}}]$).

Di seguito si riporta un frammento del metodo

```

1 x_r, y_r, theta_r = robot_pose
2
3 # ... Controllo di arrivo a destinazione basato sugli indici.../
4
5 if distance_to_goal < self.stop_tolerance:
6     if index_distance_to_end < max_index_gap:
7         # Il robot si ferma inviando velocita' zero
8         return 0.0, 0.0
9
10 # Richiama la funzione geometrica per trovare il punto di mira
11 lookahead_pt = self.find_lookahead_point(robot_pose, path)
12
13 # Calcola le componenti del vettore che punta dal robot al target
14 dx = lookahead_pt[0] - x_r
15 dy = lookahead_pt[1] - y_r
16
17 # ... Calcola la Look-ahead distance (l_d) effettiva ...
18
19 # Calcola l'angolo globale del vettore che punta al target
20 # arctan2 ne permette il calcolo corretto in base ai segni di dy e dx
21 target_angle = np.arctan2(dy, dx)
22
23 # Calcola l'angolo alfa: differenza tra la direzione del target e l'
    orientamento attuale
24 alpha = target_angle - theta_r
25
26 # Normalizza l'angolo alfa nell'intervallo [-pi, pi]
27 alpha = np.arctan2(np.sin(alpha), np.cos(alpha))
28
29 # ...se il punto si trova dietro al robot il robot ruota sul posto...
30

```

```

31 # Formula centrale del Pure Pursuit basata su alfa:
32 # Calcola la curvatura  $k = (2 * \sin(\alpha)) / l_d$ 
33 curvature = (2.0 * np.sin(alpha)) / l_d
34
35 # La velocita' lineare e' mantenuta costante
36 v = self.target_v
37
38 # La velocita' angolare ( $\omega = v * k$ ) viene calcolata e "clippata" per
39 rispettare il limite
40 omega = np.clip(v * curvature, -self.max_omega, self.max_omega)
41
42 return v, omega

```

4.4 Il modulo pure_pursuit_simulation.py

Il modulo `pure_pursuit_simulation.py` è responsabile del coordinamento e dell'esecuzione dell'ambiente di simulazione dell'algoritmo *Pure Pursuit*. La sua architettura si articola attorno a tre funzioni principali. La prima, `voxel_downsample_2d`, si occupa del sottocampionamento spaziale dei dati ed è già stata approfondita nella Sezione 1.5.2. La funzione `build_map_world` estrae invece la nuvola di punti rappresentativa degli ostacoli per generare la mappa globale di riferimento (*target*) impiegata dall'algoritmo ICP, ottimizzandone la densità informatica tramite *Voxel Downsampling*. Infine, il core della simulazione è affidato a `run_simulation`, la quale gestisce l'esecuzione integrata del Pure Pursuit e dell'ICP; questa funzione permette di analizzare il comportamento del robot sia in presenza che in assenza della logica di chiusura dell'anello (*loop closure*), valutando l'impatto di un'odometria reale affetta da rumore rispetto a una ideale.

Costruzione e Ottimizzazione della Mappa Globale (`build_map_world`)

Il metodo `build_map_world` si occupa di estrarre e sintetizzare la geometria degli ostacoli presenti nell'ambiente di simulazione, generandone una rappresentazione discreta sotto forma di nuvola di punti 2D. Questa mappa globale funge da riferimento (*target point cloud*) per l'algoritmo di allineamento e registrazione *Iterative Closest Point* (ICP).

```

1 def build_map_world(env: Environment, voxel_size: float = 0.03) -> np.
  ndarray:
2     """
3     Estrae i punti degli ostacoli per usarli come mappa globale (target)

```

```

4     per l'ICP, ottimizzandone la densità tramite Voxel Down Sampling.
5     """
6     map_points = []
7     for obstacle in env.obstacles:
8         # estrazione punti ostacolo
9         x_obs, y_obs = obstacle.exterior.xy
10        # zip crea coppie di elementi che condividono lo stesso indice
11        for x, y in zip(x_obs, y_obs):
12            map_points.append([x, y])
13
14    raw_map = np.array(map_points)
15
16    # VOXEL DOWN SAMPLING
17    # Riduciamo i punti ridondanti della mappa globale
18    filtered_map = voxel_downsample_2d(raw_map, voxel_size)
19
20    return filtered_map

```

Architettura e Ciclo Principale di Simulazione (`run_simulation`)

La funzione `run_simulation` rappresenta il fulcro operativo dell'intero framework software. Il suo compito è orchestrare l'interazione tra i moduli di percezione (LiDAR), localizzazione (Odometria, ICP, Loop Closure), controllo cinematico (Pure Pursuit) e la simulazione fisica del veicolo, permettendo la valutazione del sistema sotto diverse condizioni di rumore e configurazioni ambientali.

```

1 def run_simulation(
2     path_name: str, variant: str, use_loop_closure: bool,
3     add_odom_noise: bool, dt: float, total_steps: int,
4     # ... [altri iperparametri di configurazione] ...
5 ) -> dict:

```

L'esecuzione si articola in una fase di inizializzazione seguita da un ciclo iterativo (*main loop*), le cui fasi principali sono descritte di seguito:

1. Inizializzazione del Sistema

In base ai parametri `path_name` e `variant`, la funzione istanzia la traiettoria di riferimento e l'ambiente virtuale associato (ostacoli e limiti spaziali). Tramite la funzione `build_map_world`, viene estratta la nuvola di punti globale (*target map*). Vengono inoltre istanziati il Robot, il controllore `PurePursuitController`, il sensore `Lidar` e,

se richiesto (`add_odom_noise = True`), il modulo di iniezione del rumore odometrico `NoisyOdometry`.

2. Ciclo di Controllo e Localizzazione

L'architettura del sistema si articola in una sequenza integrata di fasi operative che guidano il robot dalla percezione al controllo.

Inizialmente, durante la fase di percezione e pre-elaborazione, il robot effettua una scansione dell'ambiente circostante tramite un sensore LiDAR simulato. La nuvola di punti locale così ottenuta viene immediatamente sottoposta a un processo di filtraggio tramite *Voxel Downsampling* (con una griglia spaziale di 3 cm), con l'obiettivo di ridurre la densità dei dati e ottimizzare il carico computazionale delle successive fasi di allineamento.

Successivamente, la fase di predizione odometrica provvede a calcolare lo spostamento relativo — espresso in termini di matrice di rotazione R_Δ e vettore di traslazione t_Δ — rispetto all'iterazione precedente. Tale stima si basa sulla posa reale nel caso ideale, oppure su quella affetta da rumore qualora le perturbazioni siano attivate. Applicando questo incremento all'ultima stima globale nota, si ottiene una prima posa predetta di natura puramente cinematica.

Ove la scansione locale presenti una quantità di punti sufficiente, si procede con la correzione *Scan-to-Map* basata sull'algoritmo *Iterative Closest Point* (ICP) per far collimare la nuvola di punti corrente con la mappa globale statica. Questa procedura integra un rigoroso meccanismo di *gating* articolato su due criteri distinti. Il primo riguarda la qualità del *fitting*, la quale valuta la convergenza geometrica dell'algoritmo verificando sia il raggiungimento di un numero minimo di corrispondenze (`min_icp_correspondences`) sia il rispetto di un tetto massimo sull'Errore Quadratico Medio ($RMSE \leq \text{max_icp_rmse}$). Il secondo criterio si concentra invece sulla plausibilità fisica della soluzione: la correzione proposta dall'ICP viene controllata, scartando qualsiasi aggiustamento che imponga discontinuità o salti posizionali e rotazionali superiori alle soglie ammissibili dinamiche `effective_max_angle` e `effective_max_pos`, i cui valori aumentano in base al numero di corrispondenze ICP trovate. In questo modo si prevengono efficacemente fenomeni di divergenza numerica o di errata associazione causati dal problema dell'apertura o *aperture problem* (problema secondo cui non si riesce a capire se e quanto ci si sta muovendo lungo una direzione priva di dettagli geometrici).

Nei contesti in cui la logica di *Loop Closure* è abilitata, il sistema esegue una fase di ottimizzazione storica valutando l'opportunità di memorizzare un nuovo *Keyframe*, ovvero una struttura dati contenente la posa stimata e la relativa scansione locale. Qualora sia trascorso un opportuno intervallo di stasi (*cooldown*), il sistema ricerca nell'archivio storico un *Keyframe* geograficamente prossimo alla stima attuale; se la

ricerca ha esito positivo, viene tentato un allineamento tra la scansione corrente e quella memorizzata nel passato. Anche in questo caso viene applicato un severo filtro di plausibilità per rigettare i falsi positivi generati da fenomeni di *perceptual aliasing*, in cui zone diverse dell'ambiente presentano caratteristiche geometriche del tutto analoghe.

In ultima analisi, sulla base della migliore stima di posa disponibile — che sia quella odometrica pura oppure quella affinata dal modulo ICP o dalla chiusura dell'anello — il controllore *Pure Pursuit* elabora i comandi di velocità lineare e angolare (v, ω). Tali segnali di controllo vengono infine inviati ai motori per determinare l'avanzamento dello stato fisico del robot per un intervallo temporale dt .

3. Criteri di Terminazione e Statistiche Finali

Il ciclo si interrompe automaticamente se si raggiunge il limite massimo di iterazioni (`total_steps`) per prevenire loop infiniti, oppure se il veicolo arriva a destinazione (comandi cinematici nulli). Particolare cura è dedicata ai tracciati chiusi (es. circuiti): la condizione di arresto diviene attiva solo dopo che il veicolo si è allontanato fisicamente (`min_leave_start_dist`) dal punto di partenza.

Al termine dell'esecuzione, la funzione restituisce un dizionario strutturato contenente la cronologia completa delle traiettorie e una suite dettagliata di metriche diagnostiche, indispensabili per la valutazione quantitativa delle prestazioni dell'algoritmo. In particolare, la struttura dati raccoglie le informazioni geometriche fondamentali del test, quali la traiettoria di riferimento pianificata ("`path`"), la traiettoria reale percorsa dal veicolo ("`robot_history`") e la traiettoria stimata risultante dalla fusione tra odometria, correzioni ICP ed eventuale modulo di *Loop Closure* ("`estimated_history`"). A queste si affiancano i parametri identificativi della simulazione, tra cui la configurazione dell'ambiente utilizzato ("`env`"), la specifica variante del tracciato adottata ("`variant`") e due flag booleani che indicano, rispettivamente, l'attivazione del rumore odometrico ("`noise_enabled`") e dell'algoritmo di *Loop Closure* ("`loop_closure_enabled`"). Infine, il dizionario fornisce i contatori di efficienza dei moduli di stima: per l'ICP vengono registrati il numero di correzioni validate ("`n_icp_accepted`") e di quelle scartate dal meccanismo di *gating* ("`n_icp_rejected`"), mentre per la logica di *Loop Closure* vengono tracciati i tentativi di chiusura del loop effettivamente accettati ("`n_loops`", pari a zero se il modulo è disabilitato) e quelli rigettati per vincoli di plausibilità fisica ("`n_loop_rejected`").

Capitolo 5

Valutazione sperimentale

5.1 Introduzione di capitolo

In questo capitolo vengono presentati e analizzati i risultati sperimentali ottenuti dall'applicazione dell'algoritmo *Pure Pursuit* all'interno dei diversi ambienti di test appositamente sviluppati. L'obiettivo dell'analisi consiste nel valutare e confrontare il comportamento del controllore al variare di parametri chiave, quali la distanza di *look-ahead* (L_d), la tipologia di odometria impiegata (ideale o affetta da rumore) e l'integrazione o meno della logica di *Loop Closure*.

Attraverso le prove effettuate confrontiamo le differenze comportamentali e prestazionali del sistema e per far ciò ci avvaliamo di alcune metriche quantitative e qualitative, le quali sono: l'errore medio di inseguimento, la somma cumulativa degli errori, il numero totale di passi di simulazione effettuati e il tipo di traiettoria percorsa dal veicolo.

5.2 Caratterizzazione dei parametri per type di percorso

Come illustrato nella Sezione 3.4, per ciascuno dei 12 percorsi considerati sono state definite tre distinte varianti di test, denominate rispettivamente **type1**, **type2** e **type3**. Tali configurazioni si differenziano per i valori assegnati alla *clearance* — ovvero lo spazio libero da ostacoli garantito attorno alla traiettoria teorica del robot —, al numero complessivo di ostacoli (**n_obstacles**) e al raggio massimo di scansione del sensore LiDAR (**r_max**), parametro quest'ultimo tarato in base alle specificità geometriche dell'ambiente.

Più precisamente, andiamo a rappresentare un ambiente privo di ostacoli attraverso la variante di tipo **type1**; proseguendo la variante di tipo **type2** rappresenta uno scenario con una bassa densità di ostacoli mentre la variante di tipo **type3** viene usata

per rappresentare un ambiente con un'elevata densità di ostacoli. Occorre notare come essendo la configurazione di tipo `type2` a minore densità di ostacoli rispetto all'ambiente di tipo `type3`, il valore di `r_max` in tale configurazione risulta maggiore rispetto alla sua controparte.

I valori di `r_max` sono stati opportunamente adattati per ciascuna tipologia di tracciato al fine di ottimizzare le prestazioni complessive del sistema di percezione e controllo.

La Tabella 5.1 riassume i valori assunti dai parametri di configurazione per ciascun tracciato e per le tre varianti di ambiente.

Tabella 5.1: Parametri di configurazione degli ambienti di test suddivisi per percorso e tipologia (*type*).

Percorso	Tipologia	Clearance [m]	N. Ostacoli	$r_{\max}$ [m]
straight_short	type1	0.8	0	4.0
	type2	0.7	2	8.0
	type3	0.6	5	4.0
straight_long	type1	0.9	0	6.0
	type2	0.9	3	12.0
	type3	0.7	10	6.0
contapassi	type1	0.9	0	6.0
	type2	0.9	3	12.0
	type3	0.7	10	6.0
circle_medium	type1	0.9	0	5.0
	type2	0.8	2	10.0
	type3	0.7	8	5.0
circle_large	type1	1.0	0	7.0
	type2	0.9	3	14.0
	type3	0.8	10	7.0
tight_slalom	type1	0.7	0	4.5
	type2	0.7	3	5.5
	type3	0.5	8	4.5
wide_slalom	type1	1.0	0	6.5
	type2	0.9	3	10.0
	type3	0.8	10	6.5
eight	type1	0.9	0	6.0
	type2	0.9	3	8.0
	type3	0.7	9	6.0
square	type1	1.0	0	6.5
	type2	0.9	4	12.5
	type3	0.8	10	6.5
pista_1	type1	1.0	0	7.0
	type2	0.9	4	14.0
	type3	0.8	10	7.0
pista_2	type1	1.0	0	6.0
	type2	0.9	5	10.0
	type3	0.8	12	8.0
pista_3_(2_giri)	type1	1.0	0	6.0
	type2	0.9	3	9.0
	type3	0.8	8	6.0

5.3 Metriche di valutazione

Per una valutazione quantitativa e qualitativa del controllore e del sistema di stima, analizziamo le seguenti metriche. Le metriche di tipo quantitativo che andiamo a valutare sono: l'Errore Medio, calcolato come la media aritmetica degli scostamenti geometrici istantanei lungo il percorso; essendo le distanze sempre positive, tale valore corrisponde formalmente all'Errore Assoluto Medio (*Mean Absolute Error, MAE*), espresso in metri. Insieme ad esso valutiamo l'Errore Cumulato Totale, il quale rappresenta la somma complessiva delle deviazioni istantanee durante la simulazione. Come ultima metrica di tipo quantitativo viene valutato il numero totale di passi di simulazione completati dal robot fino al raggiungimento del traguardo (indice dell'efficienza temporale del percorso). Come metrica qualitativa invece si osserva il profilo della traiettoria seguita dal veicolo, la quale viene utilizzato per valutare la fluidità del movimento e la stabilità del tracciamento.

5.4 Configurazione degli esperimenti

Come descritto nella Sezione 3.3.2, la campagna sperimentale è stata condotta su tutti e 12 i percorsi previsti. Per ciascun tracciato e per ogni variante d'ambiente (**type1**, **type2** e **type3**), gli esperimenti sono stati eseguiti valutando diverse condizioni operative: odometria ideale rispetto a quella affetta da rumore, applicazione dell'algoritmo ICP con e senza la logica di *Loop Closure*, e tre differenti valori della distanza di *look-ahead* (L_d).

I tre valori di distanza di *look-ahead* selezionati per le prove sono rispettivamente 0.2 m, 0.4 m e 0.6 m.

Particolare rilevanza assume la configurazione con odometria ideale all'interno degli ambienti di tipo **type1**, nei quali, data l'assenza di ostacoli, la posizione stimata dal robot coincide esattamente con la sua *ground truth* reale. Questo scenario di riferimento risulta essenziale per isolare e analizzare il comportamento cinematico "puro" del controllore *Pure Pursuit*, escludendo gli errori introdotti dai moduli di localizzazione.

Nelle sezioni seguenti verranno presentati e discussi i risultati sperimentali di maggior rilievo.

5.4.1 Esperimento *straight long*

Il percorso **straight_long** (rettilineo lungo 20 metri) risulta insieme alla sua controparte **straight_short** il tipo di percorso più semplice su cui facciamo i nostri esperimenti.

Osservando i risultati ottenuti possiamo notare come in un ambiente di tipo **type1**, con un utilizzo di un'odometria ideale, l'errore medio di inseguimento si mantiene costante a un valore di 0.0336 m per tutti e tre i valori della distanza di *look-ahead*

considerati. Proseguendo con l'analisi degli ambienti di tipo **type2** e **type3**, possiamo notare come l'errore medio rimanga comunque contenuto, attestandosi indicativamente tra 0.0350 m e 0.0380 m. Questo tipo di comportamento è dovuto al fatto che il percorso sia di tipo rettilineo.

Un fenomeno avvenuto che occorre evidenziare è il fatto che ci sono alcuni casi in cui le simulazioni con odometria rumorosa hanno registrato prestazioni leggermente superiori rispetto alla controparte ideale, come si è potuto notare nel caso di tipo **type2** con distanza di *look-ahead* pari a 0.6 m.

Tale fenomeno è dovuto alla natura stocastica delle perturbazioni applicate ai comandi di velocità del robot, per le quali le variazioni casuali introdotte possono agire favorevolmente sul sistema, compensando localmente la deviazione e correggendo in modo fortuito la traiettoria del veicolo.

Di seguito riportiamo alcuni grafici relativi agli esperimenti.

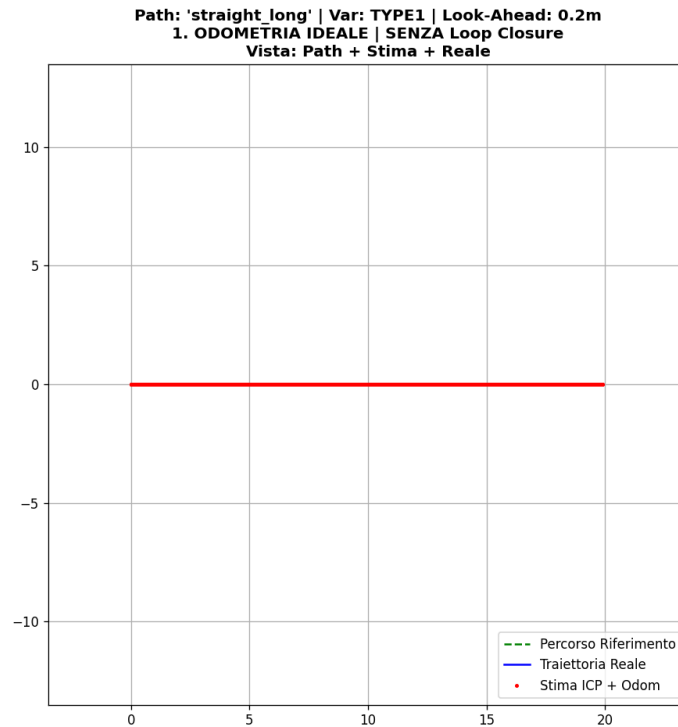


Figura 5.1: Percorso **straight_long** (TYPE1): odometria ideale senza Loop Closure con $L_d = 0.2$ m.

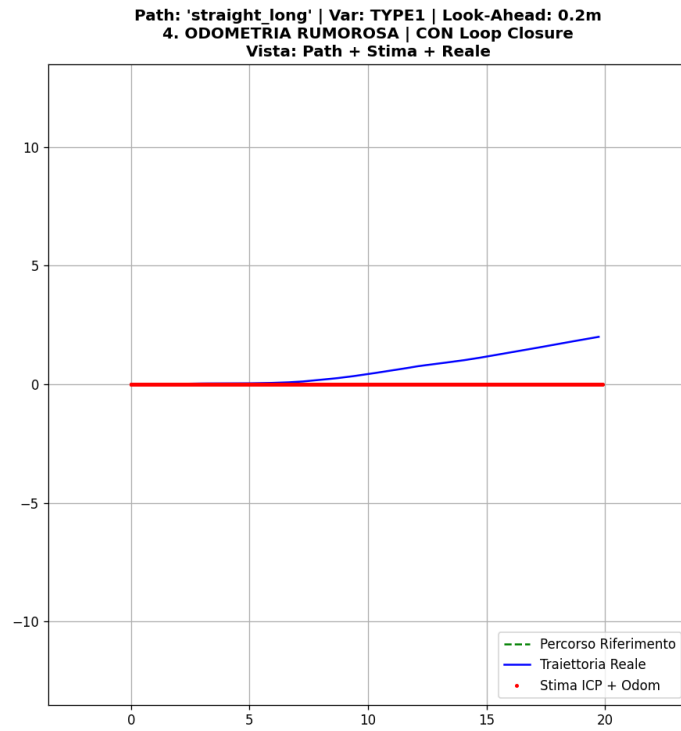


Figura 5.2: Percorso `straight_long` (TYPE1): odometria rumorosa con Loop Closure con $L_d = 0.2$ m.

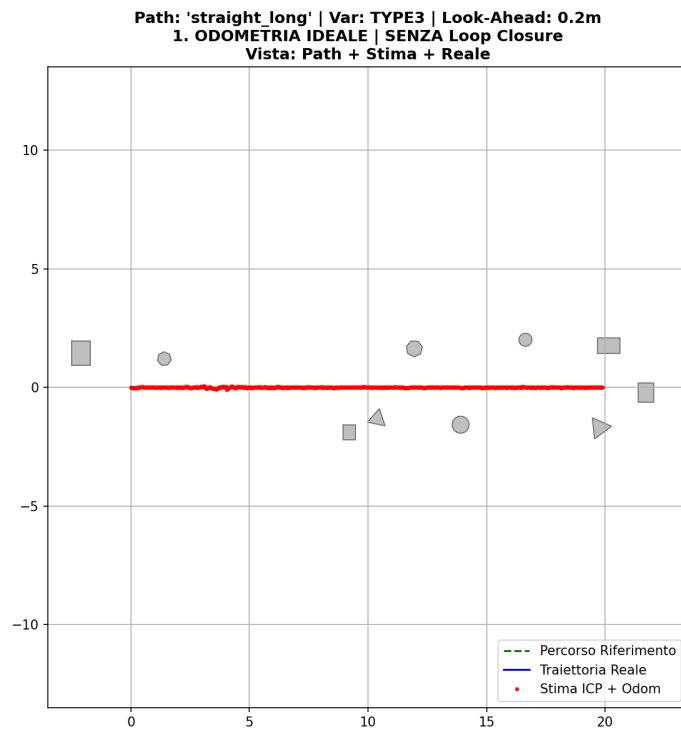


Figura 5.3: Percorso `straight_long` (TYPE3): vista completa del percorso, stima e traiettoria reale con $L_d = 0.2$ m.

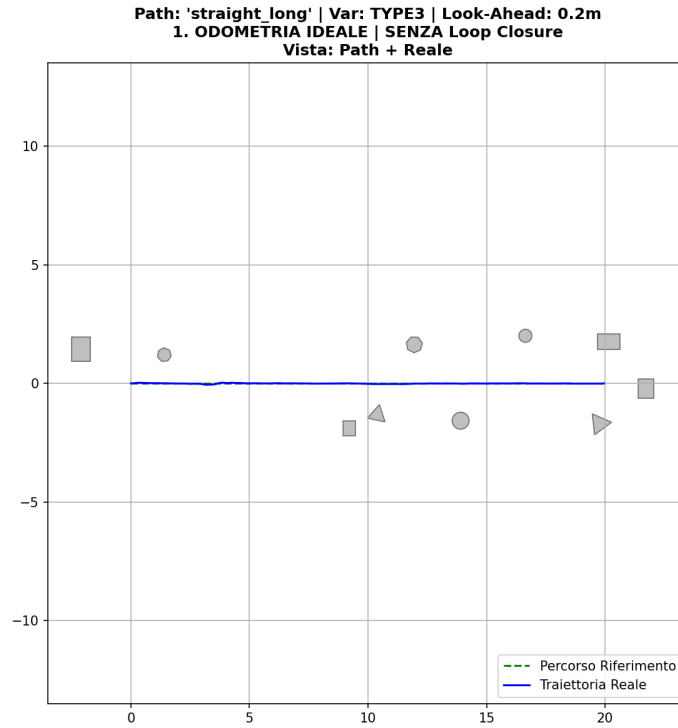


Figura 5.4: Percorso `straight_long` (TYPE3): dettaglio della traiettoria reale e disposizione degli ostacoli con $L_d = 0.2$ m.

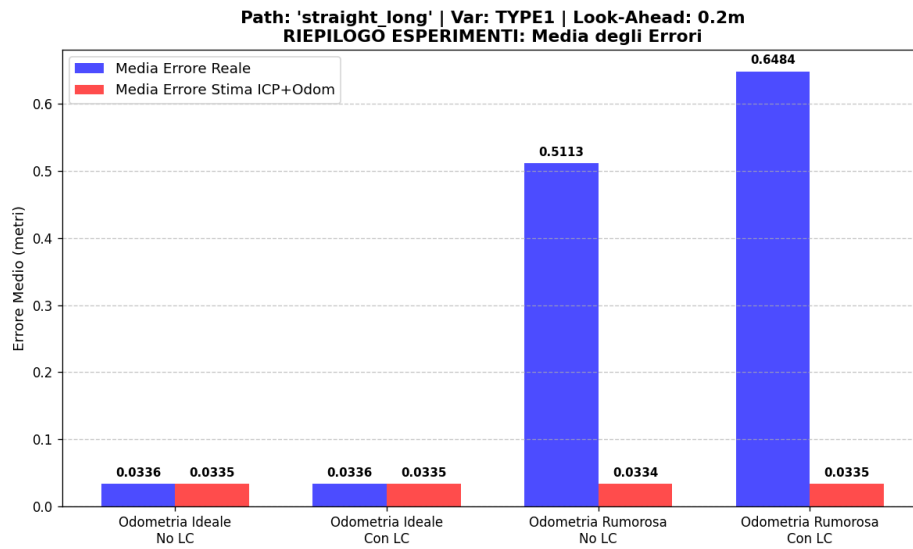


Figura 5.5: Riepilogo dell'errore medio per il percorso `straight_long` nell'ambiente TYPE1 con $L_d = 0.2$ m.

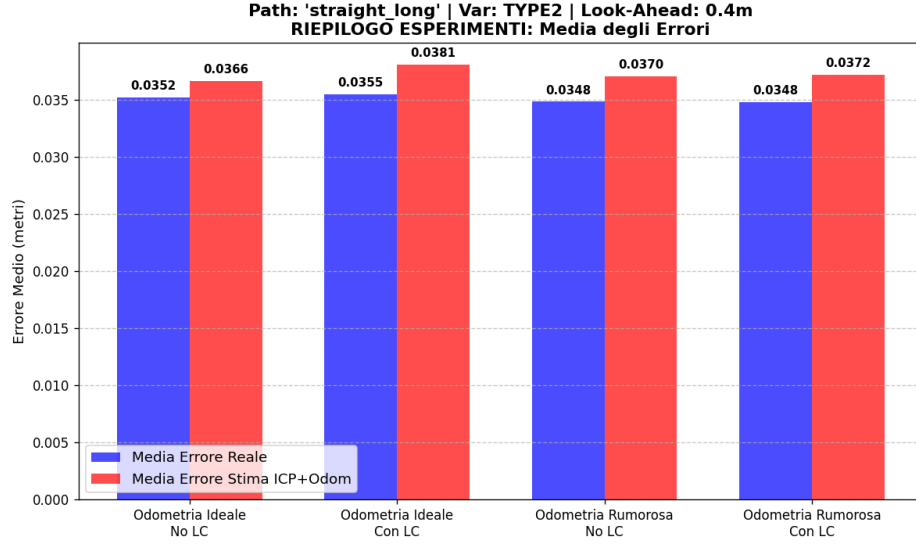


Figura 5.6: Riepilogo dell'errore medio per il percorso `straight_long` nell'ambiente TYPE2 con $L_d = 0.4$ m.

5.4.2 Esperimento *contapassi*

Il percorso *contapassi* è stato progettato per testare il comportamento del robot in uno scenario limite, caratterizzato da una sequenza ravvicinata di svolte estremamente strette. La finalità principale di questo tracciato consiste nell'analizzare la stabilità e l'accuratezza del controllore *Pure Pursuit* al variare della distanza di *look-ahead* (L_d).

Analizzando la configurazione **type1** con odometria ideale e priva di *Loop Closure*, si osserva che all'aumentare della distanza di *look-ahead* il robot tende ad anticipare progressivamente le svolte, "tagliando" i tratti curvi del percorso. Ciò comporta una contestuale riduzione del numero totale di passi di simulazione necessari per completare il tracciato, i quali passano da 6065 con $L_d = 0.2$ m, a 5777 con $L_d = 0.4$ m, fino a 5538 con $L_d = 0.6$ m.

È opportuno notare che tale fenomeno si manifesta esclusivamente nei tratti in cui il valore di *look-ahead* consente al punto di mira di proiettarsi oltre la curva. Al contrario, lungo i tratti rettilinei il comportamento del robot rimane invariato: per percorrere un segmento rettilineo di 2.4 m, il veicolo impiega esattamente 121 passi per tutti e tre i valori di L_d considerati.

Una conseguenza diretta del taglio delle curve è l'incremento dell'errore medio di tracciamento all'aumentare della distanza di *look-ahead*, che cresce progressivamente da un valore di 0.0745 m ($L_d = 0.2$ m), a 0.0829 m ($L_d = 0.4$ m), fino a raggiungere 0.0935 m ($L_d = 0.6$ m), nel caso del **type1** con odometria ideale senza loop closure.

Di seguito riportiamo alcune immagini significative.

Path: 'contapassi' | Variante: TYPE1 | Look Ahead: 0.2 m
 Gruppo [1/3] | Esp [1/5] (1. ODOMETRIA IDEALE | SENZA Loop Closure) | Vista [5/5] (Auto Animata)
1. ODOMETRIA IDEALE | SENZA Loop Closure
Vista: Auto Animata

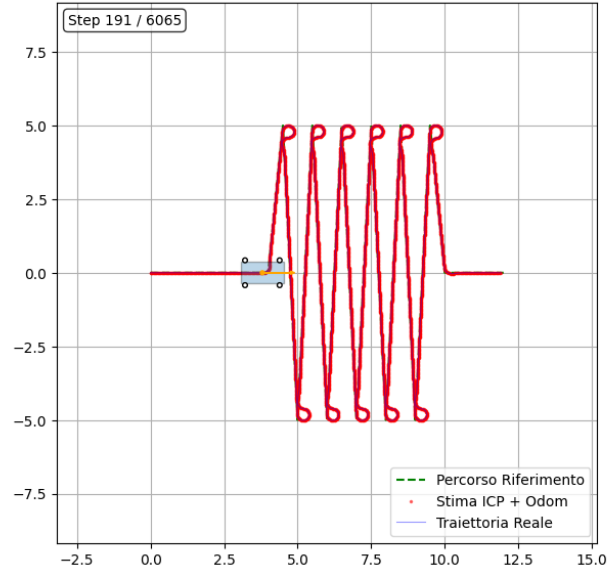


Figura 5.7: Percorso contapassi (TYPE1): odometria ideale senza Loop Closure ($L_d = 0.2$ m).

Path: 'contapassi' | Variante: TYPE1 | Look Ahead: 0.2 m
 Gruppo [1/3] | Esp [1/5] (1. ODOMETRIA IDEALE | SENZA Loop Closure) | Vista [5/5] (Auto Animata)
1. ODOMETRIA IDEALE | SENZA Loop Closure
Vista: Auto Animata

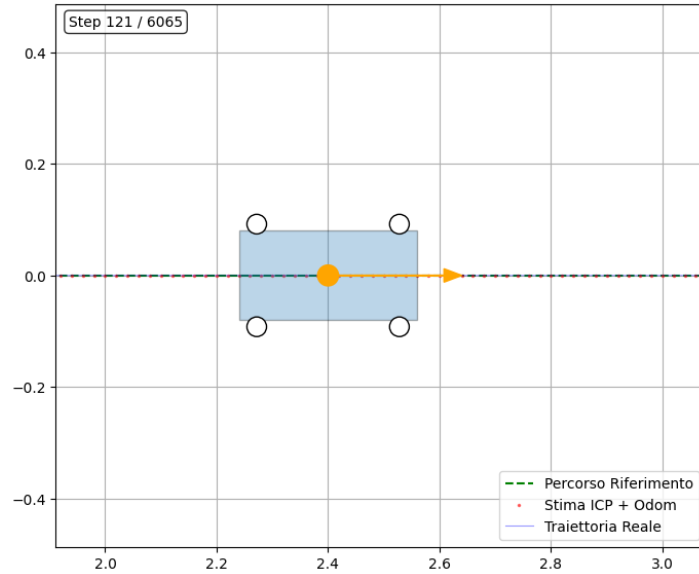


Figura 5.8: Percorso contapassi (TYPE1): dettaglio della traiettoria con odometria ideale ($L_d = 0.2$ m).

Path: 'contapassi' | Variante: TYPE1 | Look Ahead: 0.4 m
 Gruppo [2/3] | Esp [1/5] (1. ODOMETRIA IDEALE | SENZA Loop Closure) | Vista [5/5] (Auto Animata)

1. ODOMETRIA IDEALE | SENZA Loop Closure
Vista: Auto Animata

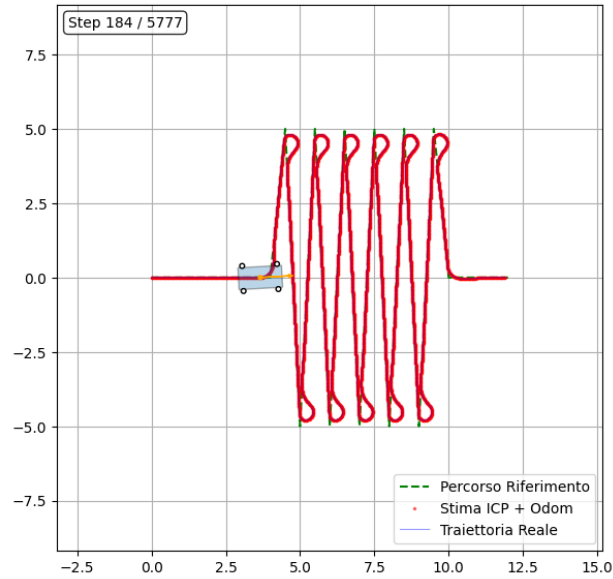


Figura 5.9: Percorso contapassi (TYPE1): odometria ideale senza Loop Closure ($L_d = 0.4$ m).

Path: 'contapassi' | Variante: TYPE1 | Look Ahead: 0.6 m
 Gruppo [3/3] | Esp [1/5] (1. ODOMETRIA IDEALE | SENZA Loop Closure) | Vista [5/5] (Auto Animata)

1. ODOMETRIA IDEALE | SENZA Loop Closure
Vista: Auto Animata

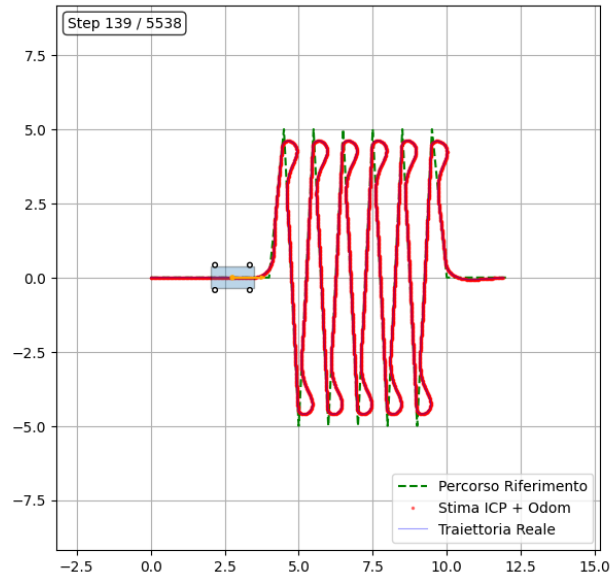


Figura 5.10: Percorso contapassi (TYPE1): odometria ideale senza Loop Closure ($L_d = 0.6$ m).

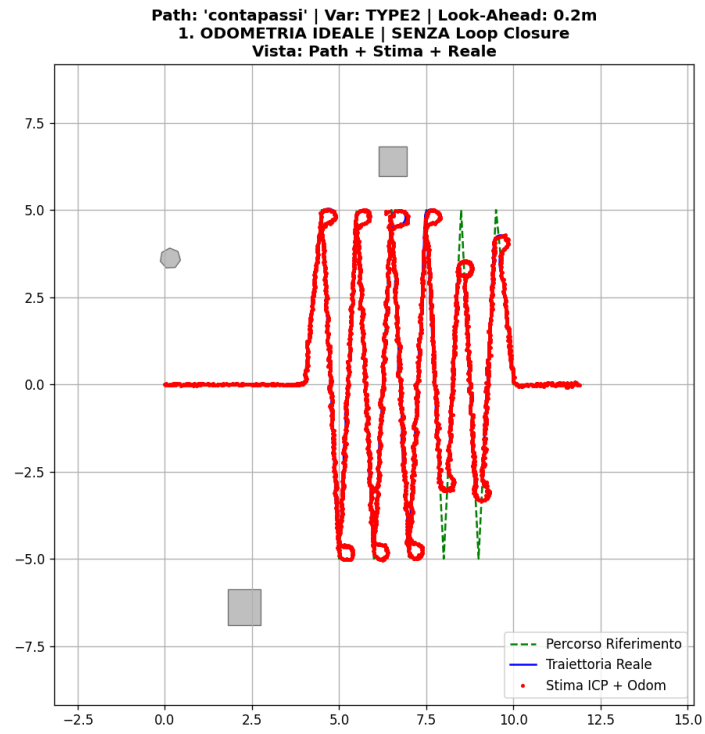


Figura 5.11: Percorso contapassi (TYPE2): vista completa con odometria ideale senza Loop Closure ($L_d = 0.2\text{m}$).

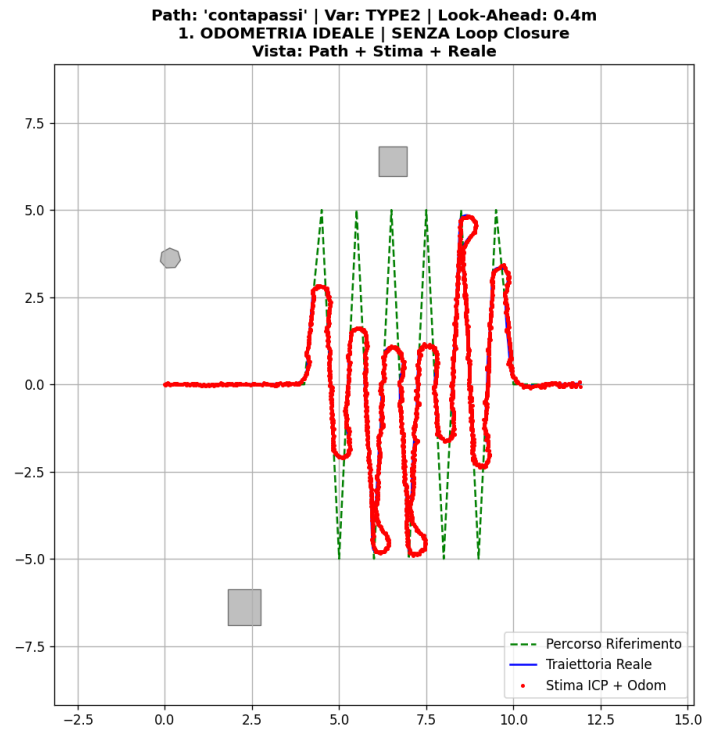


Figura 5.12: Percorso contapassi (TYPE2): vista completa con odometria ideale senza Loop Closure ($L_d = 0.4\text{m}$).

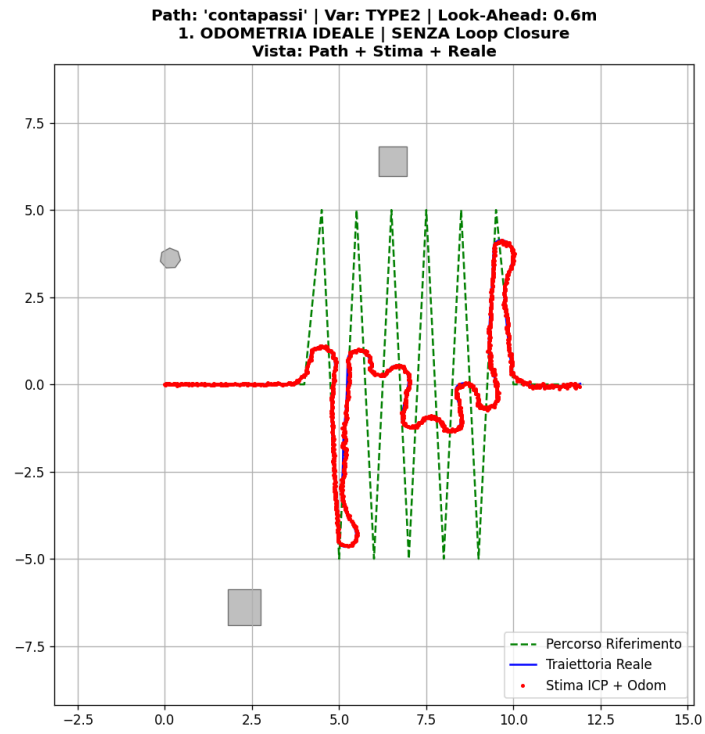


Figura 5.13: Percorso contapassi (TYPE2): vista completa con odometria ideale senza Loop Closure ($L_d = 0.6$ m).

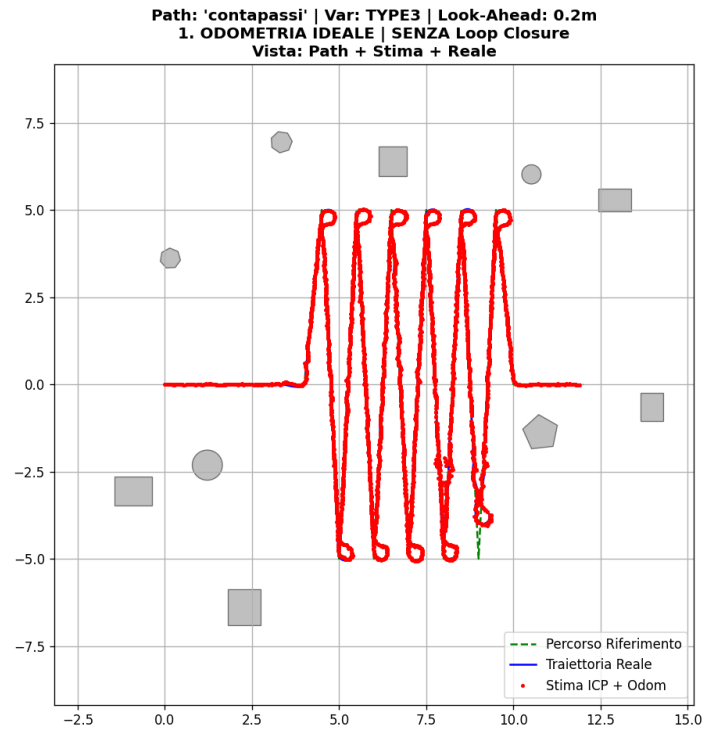


Figura 5.14: Percorso contapassi (TYPE3): vista completa con odometria ideale senza Loop Closure ($L_d = 0.2$ m).

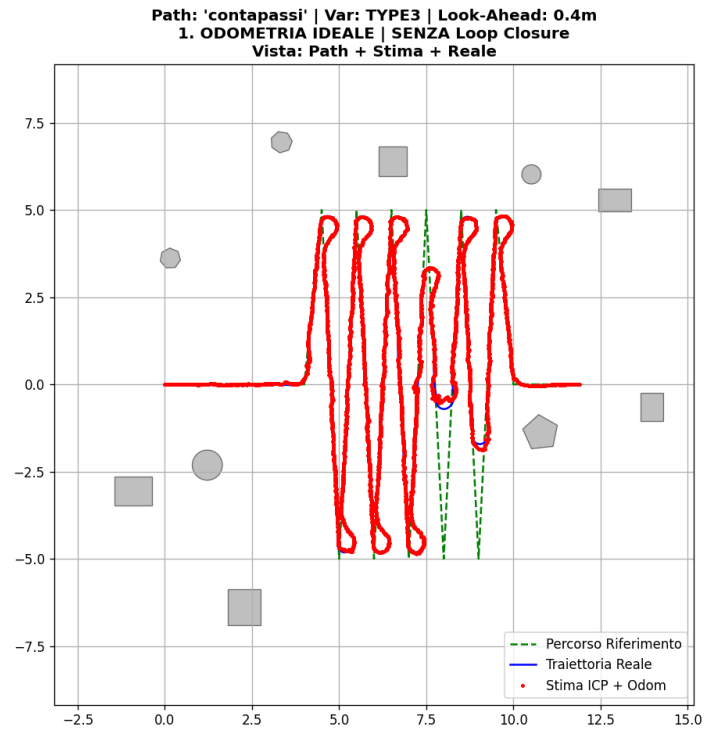


Figura 5.15: Percorso contapassi (TYPE3): vista completa con odometria ideale senza Loop Closure ($L_d = 0.4\text{m}$).

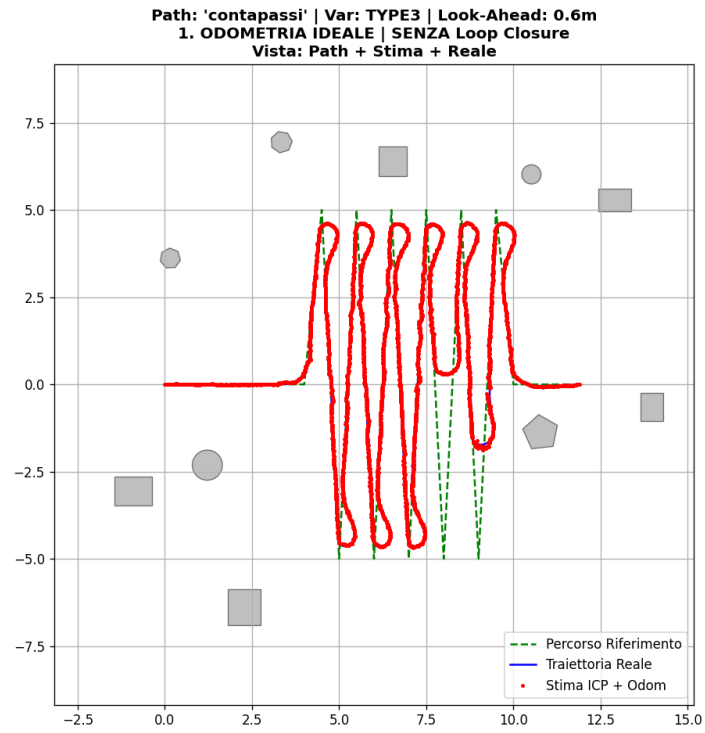


Figura 5.16: Percorso contapassi (TYPE3): vista completa con odometria ideale senza Loop Closure ($L_d = 0.6\text{m}$).

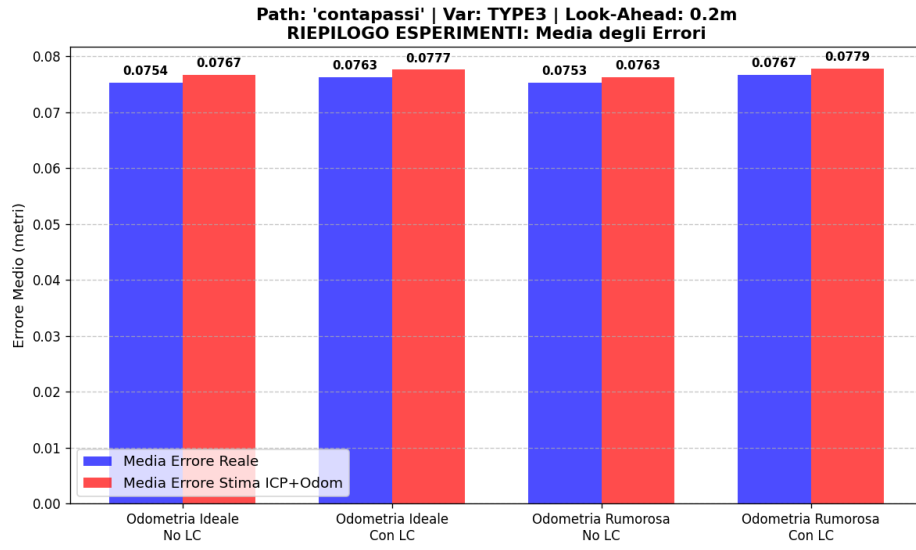


Figura 5.17: Riepilogo dell'errore medio per il percorso contapassi (TYPE3, $L_d = 0.2$ m).

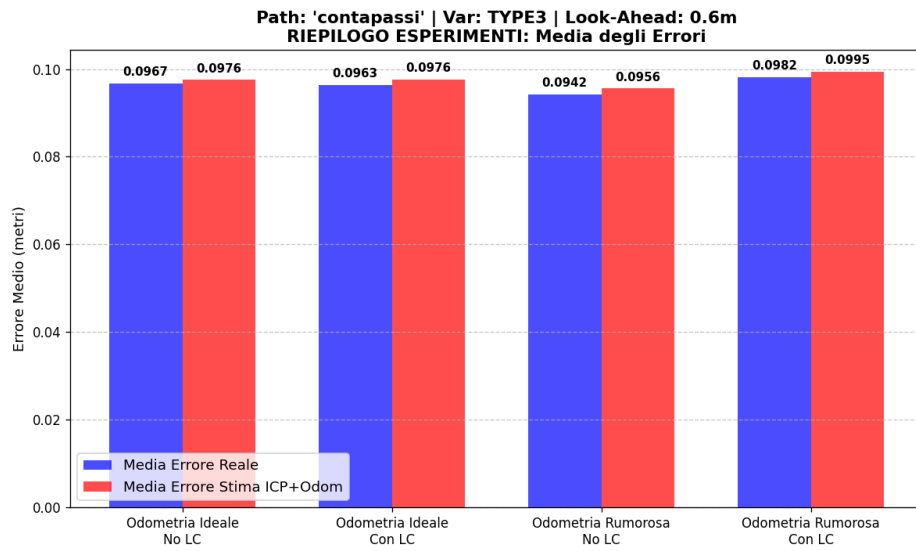


Figura 5.18: Riepilogo dell'errore medio per il percorso contapassi (TYPE3, $L_d = 0.6$ m).

5.4.3 Esperimento di tipo *pista 3 (due giri)*

In questa prova sperimentale, il tracciato consiste in un circuito caratterizzato da curve articolate, progettato per essere percorso per due giri consecutivi. Tale configurazione consente di valutare in modo approfondito il comportamento e l'efficacia dei meccanismi di *Loop Closure*.

Il circuito comprende quattro curve particolarmente strette, analoghe a quelle già analizzate nel percorso **contapassi**; anche in questo scenario si osserva che, all'aumentare della distanza di *look-ahead* (L_d), il robot tende a "tagliare" le svolte.

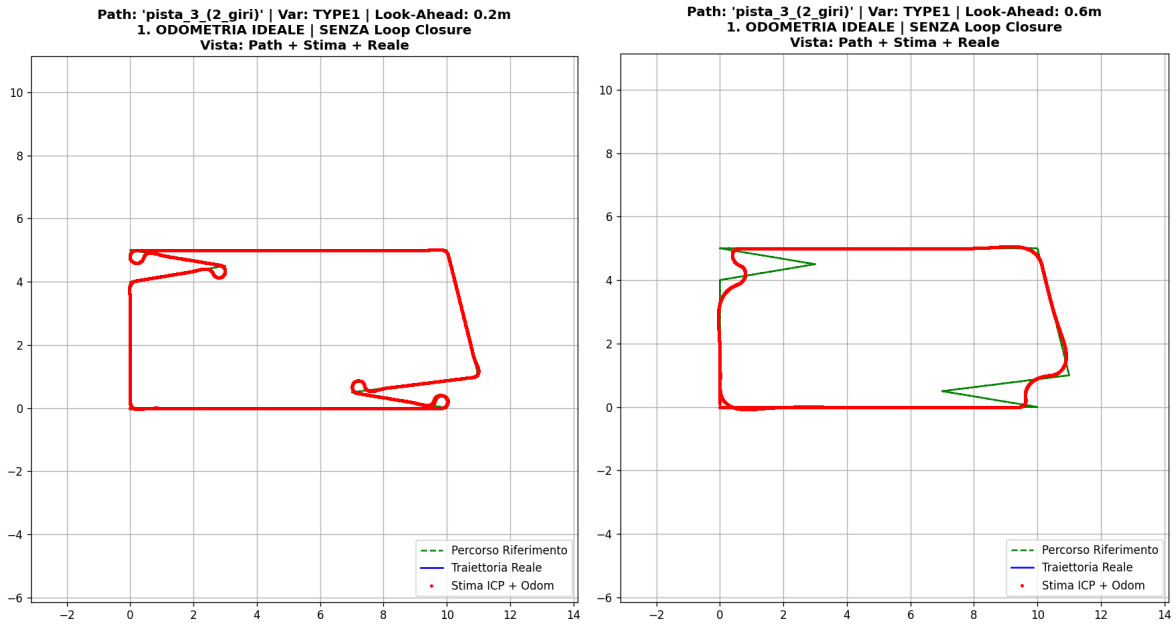
Per quanto riguarda l'apporto del *Loop Closure* — la cui efficacia può emergere appieno grazie alla percorrenza del doppio giro — si riscontrano i seguenti comportamenti al variare dell'ambiente di simulazione:

- **Ambiente TYPE1:** L'errore medio rimane invariato. Trattandosi di un ambiente privo di ostacoli, la localizzazione si basa unicamente sull'odometria, rendendo ininfluente il meccanismo di chiusura del ciclo.
- **Ambiente TYPE2:** I test sono stati condotti valutando due differenti portate massime del sensore LiDAR ($r_{\max}$): 9.0 m (valore specifico ottimizzato per questo ambiente) e 6.0 m (valore di default impiegato nell'ambiente TYPE3), al fine di evidenziarne le differenze.
 - Con $r_{\max} = 9.0$ m, l'impiego del *Loop Closure* produce un chiaro miglioramento nei casi con $L_d = 0.2$ m e $L_d = 0.4$ m, mentre porta a un peggioramento delle prestazioni quando $L_d = 0.6$ m.
 - Riducendo la portata a $r_{\max} = 6.0$ m, si osservano miglioramenti per $L_d = 0.2$ m e $L_d = 0.4$ m esclusivamente in presenza di odometria ideale. Al contrario, con $L_d = 0.6$ m si registra un miglioramento solo nel caso di odometria rumorosa.
- **Ambiente TYPE3:** Con $L_d = 0.2$ m i risultati sono pressoché identici con o senza *Loop Closure*, presentando variazioni trascurabili. Con $L_d = 0.4$ m si nota un miglioramento limitato al caso di odometria ideale, mentre con $L_d = 0.6$ m l'attivazione del *Loop Closure* peggiora le prestazioni complessive.

Dalle prove effettuate emerge chiaramente come i risultati ottenuti in presenza di odometria rumorosa siano soggetti a una componente stocastica rilevante. Per una caratterizzazione esaustiva e statisticamente solida di questo fenomeno, si renderebbe necessaria una campagna di test su un campione numericamente esteso (nell'ordine delle 100 esecuzioni per ciascuna configurazione).

Infine, occorre evidenziare che la calibrazione di $r_{\max}$ nei diversi ambienti è stata condotta con l'obiettivo di massimizzare le prestazioni complessive. Nell'ambiente TYPE2, infatti, il passaggio da $r_{\max} = 9.0\text{ m}$ a $r_{\max} = 6.0\text{ m}$ determina un netto incremento dell'errore medio di tracciamento.

Di seguito si riportano i grafici maggiormente significativi espressi in questa fase sperimentale.



(a) Odometria Ideale senza Loop Closure ($L_d = 0.2\text{ m}$). (b) Odometria Ideale senza Loop Closure ($L_d = 0.6\text{ m}$).

Figura 5.19: Percorso `pista_3` a doppio giro (TYPE1): confronto tra l'inseguimento con $L_d = 0.2\text{ m}$ e $L_d = 0.6\text{ m}$ in condizione di odometria ideale. Si nota chiaramente come l'aumento della distanza di *look-ahead* porti il robot a "tagliare" accentuatamente le curve del circuito.

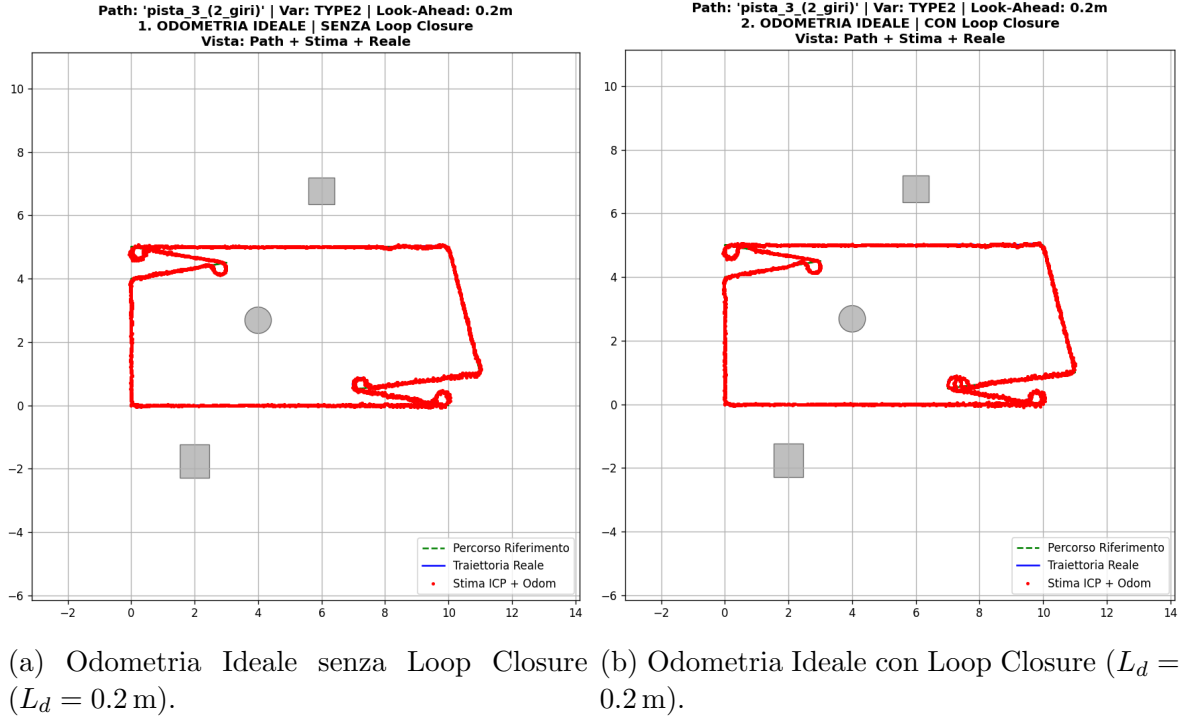


Figura 5.20: Percorso `pista_3` in ambiente con ostacoli (TYPE2) con portata LiDAR $r_{\max} = 9.0$ m: confronto dell'effetto dell'attivazione del *Loop Closure* con $L_d = 0.2$ m e odometria ideale.

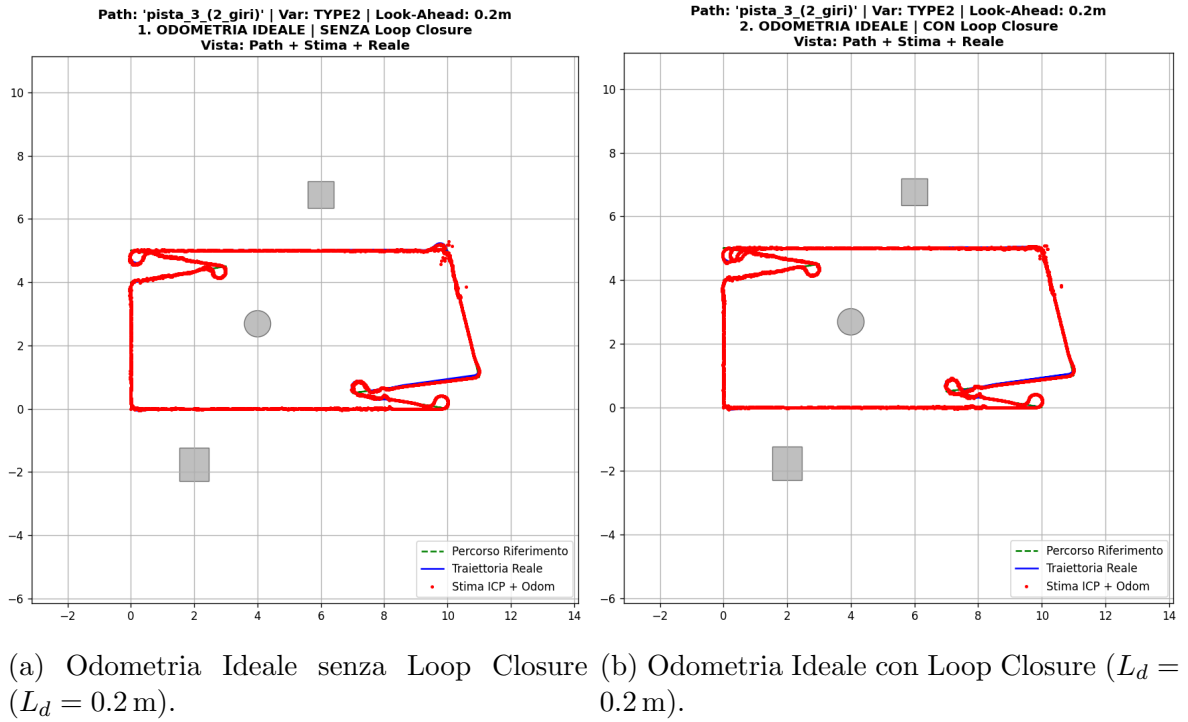


Figura 5.21: Percorso `pista_3` in ambiente con ostacoli (TYPE2) con portata LiDAR ridotta a $r_{\max} = 6.0$ m: valutazione delle prestazioni del *Loop Closure* con $L_d = 0.2$ m in odometria ideale.

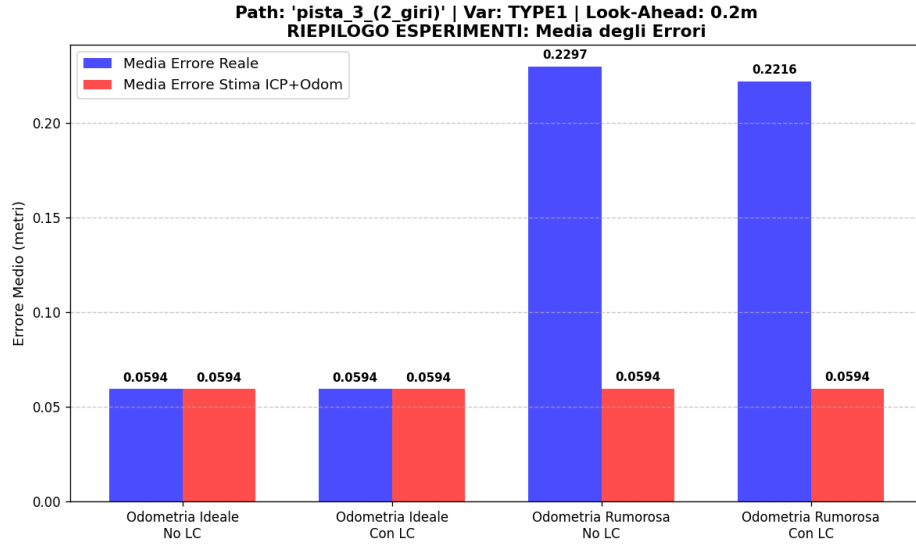


Figura 5.22: Percorso *pista_3* (TYPE1): riepilogo dell'errore medio di inseguimento con $L_d = 0.2$ m. In questo ambiente privo di ostacoli, i valori ottenuti in presenza di odometria ideale risultano identici sia con che senza l'impiego del *Loop Closure*.

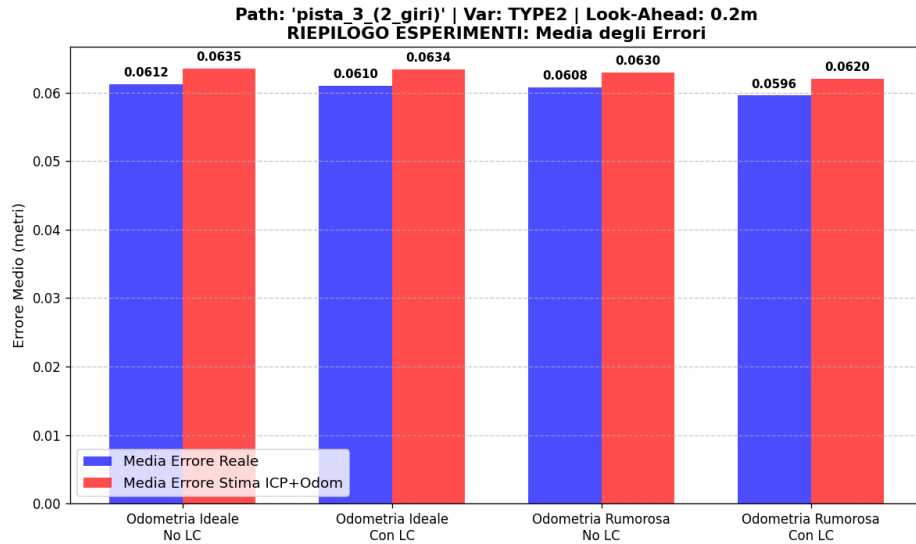


Figura 5.23: Percorso *pista_3* (TYPE2): riepilogo dell'errore medio con $L_d = 0.2$ m e portata del LiDAR $r_{\max} = 9.0$ m. In questa configurazione, l'impiego del *Loop Closure* porta sempre dei vantaggi in termini di riduzione dell'errore. Inoltre, rispetto al caso con $r_{\max} = 6.0$ m, si registra una media degli errori complessivamente più bassa.

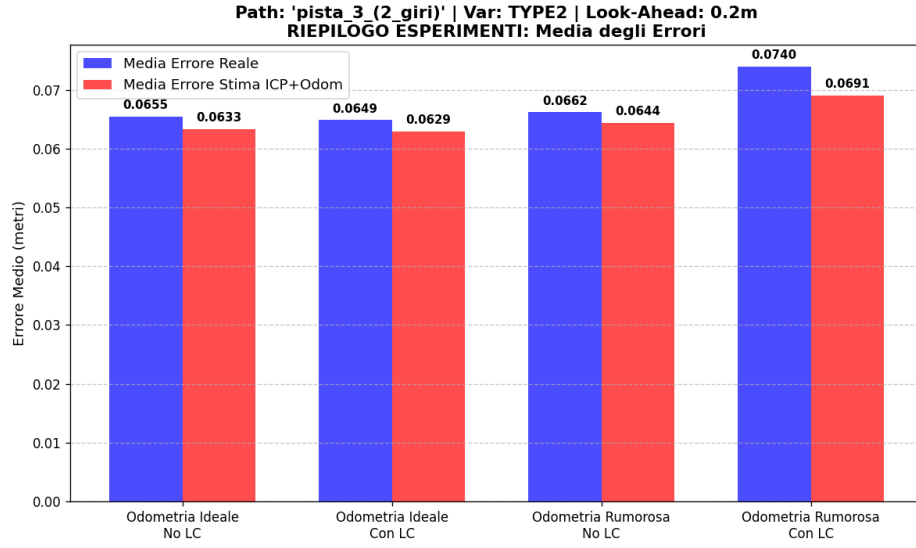


Figura 5.24: Percorso `pista_3` (TYPE2): riepilogo dell'errore medio con $L_d = 0.2$ m e portata del LiDAR ridotta a $r_{\max} = 6.0$ m. A differenza della configurazione con $r_{\max} = 9.0$ m, l'attivazione del *Loop Closure* garantisce benefici unicamente nei casi con odometria ideale.

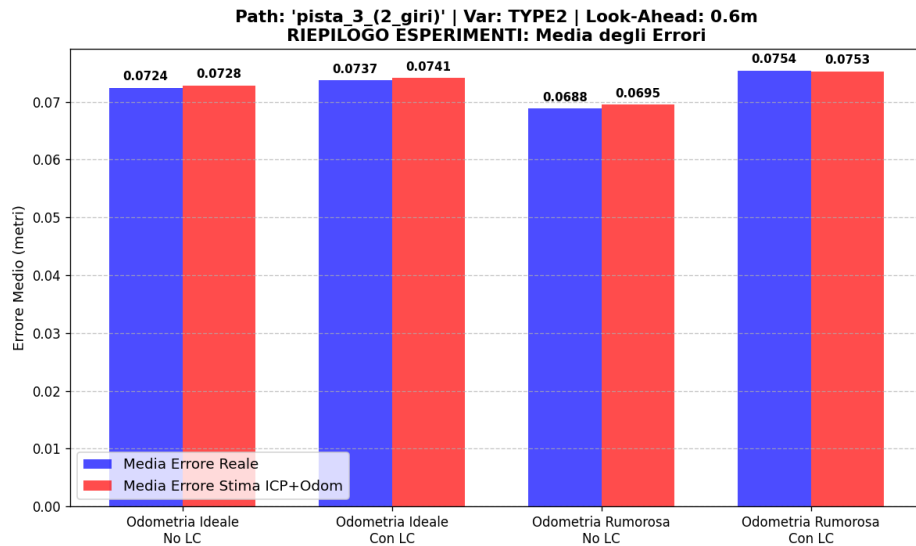


Figura 5.25: Percorso `pista_3` (TYPE2): riepilogo dell'errore medio con $L_d = 0.6$ m e portata del LiDAR $r_{\max} = 9.0$ m. In questo scenario, l'introduzione del *Loop Closure* ha condotto esclusivamente a un peggioramento delle prestazioni complessive.

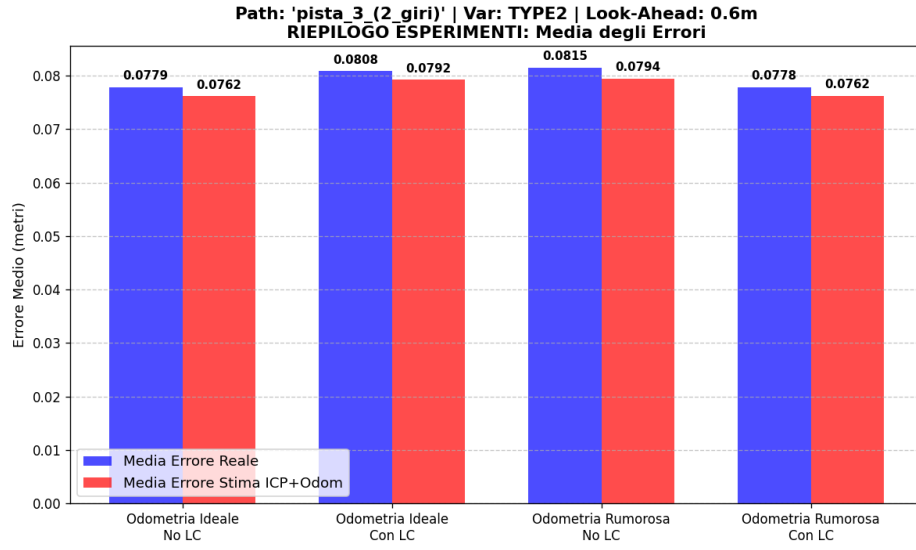


Figura 5.26: Percorso pista_3 (TYPE2): riepilogo dell'errore medio con $L_d = 0.6$ m e portata del LiDAR ridotta a $r_{\max} = 6.0$ m. In questa configurazione, l'impiego del *Loop Closure* ha mostrato miglioramenti nella sola condizione di odometria rumorosa; tuttavia, tale risultato potrebbe essere attribuibile alla natura stocastica e casuale del rumore odometrico applicato.

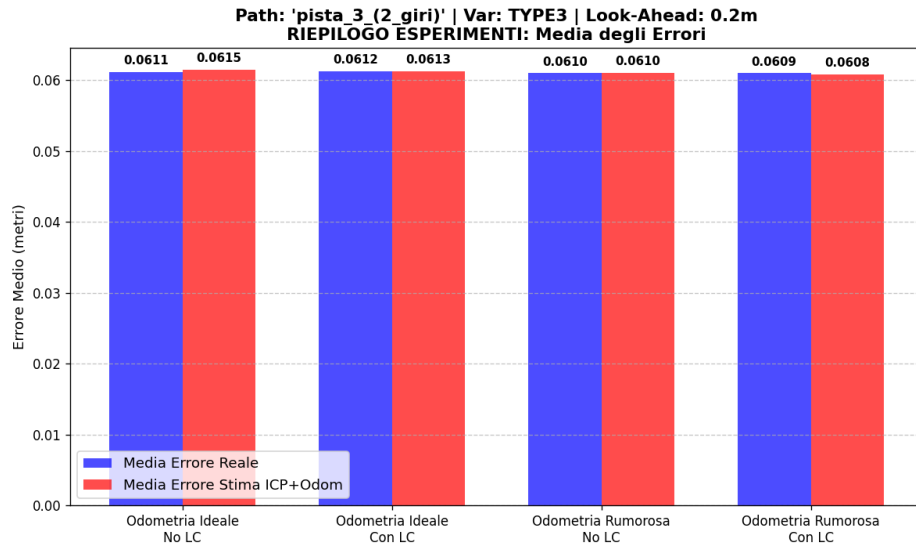


Figura 5.27: Percorso pista_3 (TYPE3): riepilogo dell'errore medio di inseguimento con $L_d = 0.2$ m. In questa configurazione, le variazioni registrate tra la presenza e l'assenza del *Loop Closure* risultano pressoché infinitesimali.

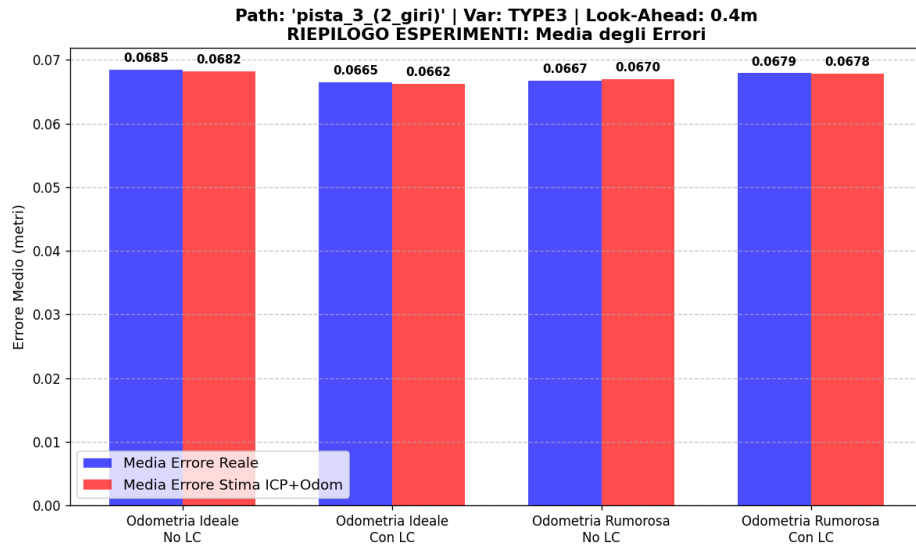


Figura 5.28: Percorso `pista_3` (TYPE3): riepilogo dell'errore medio di inseguimento con $L_d = 0.4$ m. Con un valore di *look-ahead* maggiore, l'attivazione del *Loop Closure* porta a un chiaro miglioramento delle prestazioni nel caso di odometria ideale.

Capitolo 6

Conclusioni e sviluppi futuri

6.1 Riassunto degli esperimenti

In questa tesi è stato implementato l'algoritmo di *Pure Pursuit*, il quale ha il compito di generare dei comandi di guida corretti permettendo a un robot di seguire una traiettoria prestabilita. Come viene descritto dettagliatamente nei capitoli precedenti, il controllo è basato sulla conoscenza della posa stimata del robot — mediante i suoi sensori di bordo —, il cui punto di riferimento viene posizionato convenzionalmente nel centro geometrico del robot.

A partire da tale centro, si definisce un'area di visione circolare il cui raggio corrisponde alla distanza di *look-ahead*. L'algoritmo individua il punto target dall'intersezione tra questa circonferenza e il percorso pianificato, calcolando di conseguenza i comandi di velocità angolare necessari a indirizzare il robot verso il punto individuato.

Sono stati progettati dodici tracciati di test, per ognuno di essi sono presenti tre tipi diversi di configurazioni ambientali (ognuna avente una diversa distribuzione degli ostacoli), così da permettere un'analisi dettagliata e rigorosa delle prestazioni. Per ogni tipo di test, gli esperimenti sono stati condotti valutando tre distinti valori della distanza di *look-ahead* (0.2 m, 0.4 m e 0.6 m), considerando sia la condizione di odometria ideale che rumorosa, e analizzando infine l'impatto dell'abilitazione o meno del modulo di *Loop Closure*.

In tutti gli esperimenti sono state valutate la traiettoria percorsa dal robot, la durata complessiva della simulazione espressa in numero di passi, l'errore medio — calcolato come la media aritmetica degli scostamenti geometrici istantanei lungo il percorso — e l'errore cumulato totale, che sintetizza la somma di tali deviazioni per l'intera esecuzione.

Tra le prove effettuate, si è ritenuto opportuno mostrare e analizzare nel dettaglio i seguenti esperimenti significativi.

Nell'esperimento relativo al tracciato rettilineo di 20 metri (*straight long*), si osserva

come, all'interno dell'ambiente privo di ostacoli, l'errore medio si mantenga costante per tutti e tre i valori di *look-ahead* considerati.

Inoltre, si sono evidenziati alcuni scenari in cui l'odometria rumorosa ha mostrato prestazioni inaspettatamente superiori rispetto alla controparte ideale. Tale fenomeno è riconducibile alla natura stocastica delle perturbazioni applicate ai comandi di velocità del robot: le variazioni casuali introdotte possono infatti agire favorevolmente sul sistema, compensando localmente le deviazioni e correggendo in modo fortuito la traiettoria del veicolo.

L'esperimento relativo al tracciato *contapassi* è stato progettato per testare il comportamento del robot in uno scenario limite, caratterizzato da una sequenza ravvicinata di svolte estremamente strette. La finalità principale di questo percorso consiste nell'analizzare la stabilità e l'accuratezza del controllore *Pure Pursuit* al variare della distanza di *look-ahead* (L_d).

Dalle prove effettuate si osserva come, all'aumentare della distanza di *look-ahead*, il robot tenda ad anticipare progressivamente le svolte, "tagliando" le traiettorie in curva. Ciò comporta una contestuale riduzione del numero totale di passi di simulazione necessari per completare il tracciato. È opportuno notare che tale fenomeno si manifesta esclusivamente nei tratti in cui il valore di *look-ahead* consente al punto di mira di proiettarsi oltre la curva; al contrario, lungo i tratti rettilinei il comportamento del veicolo rimane invariato. Una conseguenza diretta del taglio delle curve è il progressivo incremento dell'errore medio di tracciamento al crescere della distanza L_d .

Nell'esperimento relativo alla *pista 2* sono stati compiuti due giri consecutivi sul medesimo tracciato al fine di valutare i benefici introdotti dal modulo di *Loop Closure*. Per l'ambiente di tipo 2 (caratterizzato da una bassa densità di ostacoli), la prova è stata estesa confrontando due differenti configurazioni della portata massima (*range*) del sensore LiDAR.

Poiché l'ambiente di tipo 2 presenta un numero ridotto di ostacoli, la portata nominale del LiDAR risulta originariamente superiore rispetto a quella impostata per scenari più complessi (ambiente di tipo 3). È stato quindi eseguito un test comparativo applicando all'ambiente 2 la medesima portata ridotta prevista per l'ambiente 3, al fine di osservarne l'impatto prestazionale.

I risultati mostrano che l'impiego della *Loop Closure* si rivela efficace per distanze di *look-ahead* pari a 0.2m e 0.4m, sebbene siano stati riscontrati casi in cui essa non apporta miglioramenti significativi o persino determina un peggioramento delle prestazioni.

Emerge inoltre chiaramente come i risultati ottenuti in presenza di odometria rumorosa siano soggetti a una rilevante componente stocastica; pertanto, per una caratterizzazione esaustiva e statisticamente solida di tale fenomeno, si renderebbe necessaria una campagna di test su un campione numericamente più esteso.

Infine, si evidenzia come la riduzione della portata del sensore LiDAR all'interno dell'ambiente di tipo 2 abbia determinato un generale decadimento delle prestazioni del sistema.

6.2 Conclusioni

Dagli esperimenti descritti nel paragrafo precedente sono state dedotte le seguenti conclusioni.

Come primo tema risalta come la distanza di *look-ahead* (L_d) rappresenti un parametro fondamentale nella dinamica di movimento.

All'aumentare di tale parametro, si osserva una marcata tendenza del robot a tagliare le curve e parti del percorso; ciò comporta un incremento dell'errore medio di inseguimento della traiettoria, pur ottenendo in molti casi una riduzione del numero totale di passi di simulazione necessari per completare il percorso. L'introduzione della *search window* si è rivelata una contromisura efficace per mitigare questo fenomeno, limitando la ricerca dei punti di mira lungo il tracciato.

L'utilizzo del meccanismo di *Loop Closure* è risultato sia favorevole che sfavorevole a seconda del contesto nel quale è stato impiegato. Nei contesti più favorevoli esso ha portato a una riduzione dell'errore medio e a un miglioramento dell'accuratezza globale, ma al contempo sono stati osservati dei contesti meno favorevoli in cui l'impiego del meccanismo ha portato a un peggioramento delle prestazioni. Dai risultati è possibile comprendere che l'efficacia del meccanismo dipende dalle caratteristiche dell'ambiente in cui viene impiegato, dalla quantità di rumore presente nell'odometria e dalla distanza di *look-ahead* impostata.

Come è intuibile, la portata massima del sensore LiDAR ($r_{\max}$) risulta essere un fattore determinante nel condizionare le prestazioni complessive del sistema. Infine, occorre ricordare come, nei test effettuati utilizzando un'odometria rumorosa, la presenza di una marcata componente stocastica nel processo impone di effettuare i test per un numero elevato di iterazioni, allo scopo di ottenere dei risultati statisticamente coerenti con uno scenario realistico.

6.3 Sviluppi futuri

Prima di poter esporre i possibili sviluppi futuri, occorre ricordare come le traiettorie attuali su cui sono stati eseguiti gli esperimenti riguardanti l'algoritmo di *Pure Pursuit* siano esenti da ostacoli che si intersecano con esse. Infatti, nell'implementazione attuale non sono presenti meccanismi di evasione o cambio di percorso nel caso in cui un ostacolo imprevisto sia posizionato lungo il percorso.

Per superare questo limite, potremmo indirizzare lo sviluppo futuro del sistema verso l'integrazione di un modulo di *Path Replanning* (ripianificazione dinamica del percorso).

In tal caso sarà presente un modulo di tipo *Path Planning*, il quale avrà il compito di pianificare il percorso preimpostato inizialmente, definendo la rotta ottimale tra il punto di partenza e il *target*, sulla base di una mappa statica nota; esso collaborerà con il modulo di *Path Replanning* il cui compito sarà quello di intervenire in tempo reale per ricalcolare la traiettoria qualora l'ambiente subisca modifiche dinamiche o compaiano ostacoli non previsti.

Infine si potrebbe testare il sistema su una piattaforma robotica reale, così attraverso il comportamento osservato si potrebbe valutare l'efficacia e la robustezza dell'algoritmo in un contesto operativo concreto.